

Szakképesítés megnevezése: Szoftverfejlesztő és -tesztelő

Azonosító száma: 5 0613 12 03

VIZSGAREMEK

LMS (Library Management System)

Készítették:

Tóth Zoltán András

Vágvölgyi Máté

Zsömbörgi Soma István

Osztály: 13.C

Osztály: 13.C

Osztály: 13.C

Oktatási azonosító: 72602085264

Oktatási azonosító: 72572583241

Oktatási azonosító: 72572173468

Békéscsaba, 2024/2025

Tartalomjegyzék

Bevezető.....	5
A probléma rövid ismertetése	5
Téma indoklása	5
Téma kifejtése	5
Felhasználói dokumentáció	6
Backend.....	6
Bevezetés:.....	6
Api/login (POST).....	6
Api/register (POST)	7
Api/verify-account (POST).....	9
Api/forgot-password (POST).....	9
Api/user (GET).....	11
Api/user (PUT).....	11
Api/user (DELETE)	13
Api/finalize-registration (PUT).....	13
Api/change-password (PUT).....	15
Api/top-borrowings/{limit} (GET)	16
Api/borrowings (GET)	17
Api/available-books (GET).....	18
Api/upload-img (POST).....	18
img/{ISBN} (GET)	19
Api/books (GET).....	21
Api/all-(books, users, borrowings) (GET)	22
Api/reserve (GET).....	22
Api/reserve (POST).....	23
Api/reserve/{ISBN} (DELETE)	23
Tesztelés	24
Frontend	25
Weboldal célja:.....	25
Rendszerkövetelmények:	25
Weboldal futtatása:.....	25
Kezdőlap (Home)	25
Felhasználói hitelesítés.....	25

Könyvek böngészése és szűrése (Books)	26
Felhasználói irányítópult (Dashboard)	27
Hibakezelés	27
Regisztráció folyamata:	27
Asztali alkalmazás (LMS Desktop)	32
Bejelentkezési felület (Login page)	32
Műszerfal (Dashboard)	32
Könyvek (Books)	34
Kölcsönzések (Borrowings)	35
Foglalások (Reservations)	37
Kategóriák (Categories)	39
Szerzők (Authors)	41
Kiadók (Publishers)	42
Felhasználók (Users)	44
Fejlesztői dokumentáció	46
Adatbázis	46
Books tábla	46
Publishers tábla	46
Categories tábla	47
Books_Categories tábla	47
Authors tábla	47
Books_Authors tábla	47
Roles tábla	47
Users tábla	47
Borrowings tábla	48
Reservations tábla	48
Borrowings_storage tábla	48
after_borrowings_insert trigger	48
after_borrowings_update trigger	48
delete_authors_and_books trigger	49
delete_categories_and_books trigger	49
delete_expired_reservation event	49
Backend	50
Router.php	50
Helper.php	51

SendEmail.php	51
BooksTable.php.....	52
BorrowingsTable.php.....	53
ReservationTable.php.....	53
Table.php.....	54
UserTable.php	55
BooksController.php	56
UserController.php.....	57
Response.php.....	58
User.php	59
Model.php	60
Image.php	61
Books.php.....	62
Borrowing.php.....	63
ReservationController.php	63
ImageController.php	64
BorrowingsController.php.....	65
Token.php.....	66
Router.php	66
Frontend	69
App.jsx:	69
AuthProvider.jsx	70
setAuthToken.jsx.....	72
PrivateRoute.jsx	72
Header.jsx.....	73
TopBorrowings.jsx.....	74
TopCards.jsx	75
MainCardContainer.jsx	75
Cards.jsx.....	76
FormCards.jsx	76
DashboardHeader.jsx	76
Books.jsx	77
Borrowings.jsx	78
ChangePassword.jsx.....	78
Dashboard.jsx.....	78

FinalizeRegistration.jsx.....	79
ForgotPassword.jsx	79
Home.jsx	79
Login.jsx.....	80
NotFound.jsx	80
Profile.jsx	81
Register.jsx.....	81
Reservations.jsx.....	81
Verify.jsx.....	82
Frontend Tesztelés:	84
login.test.js:	84
register.test.js:.....	84
LMS Desktop (asztali alkalmazás).....	85
Connection osztály	85
HandleFiles osztály	85
HandleQueries osztály.....	85
Egyéb osztályok	85
Login form.....	86
Main form.....	86
Profile Settings	87
Books.....	87
Borrowings	88
Reservations	88
Categories.....	89
Authors	89
Publishers	89
Users.....	90
SQL Lekérdezések	90
Tesztelés	92
Források.....	93
Backend.....	93
Frontend	93
LMS Desktop	93
Ábrajegyzék	94

Bevezető

A probléma rövid ismertetése

Többektől hallottunk panaszt a jelenlegi könyvtári rendszerre, amit többek között a mi iskolánk is használ. Ezenfelül mi, diákok is ezen a rendszeren keresztül tudjuk megtekinteni a jelenleg nálunk lévő könyvtári könyveket, és nekünk is feltűnt, hogy a jelenlegi rendszer elavult és kényelmetlen. Tehát a probléma egy régi, nem felhasználóbarát könyvtári rendszer szélesebb körű használata.

Téma indoklása

A korábban említett elavult könyvtári rendszer leváltását céloztuk meg, ehhez készítettünk egy saját szoftvert. A fejlesztést megelőzően megkérdeztünk néhány könyvtári dolgozót, hogy mit gondolnak a régi rendszerről, és mit szólnának hozzá, ha lenne egy sokkal jobb alternatíva. Többségük szívesen használna egy modernizált változatot, így hát az ő véleményüket figyelembe véve megterveztünk egy hasonló funkciókkal rendelkező programot, amit végül el is készítettünk.

Téma kifejtése

A projekt négy fő részből áll: Adatbázis, Backend, Frontend és egy Asztali alkalmazás.

Az Adatbázisban tároljuk el az összes hosszú távon szükséges információt. (Pl.: könyvek, kölcsönzések és felhasználók)

A Backend feladata az adatbázis és a Frontend közötti kapcsolat kezelése, amit PHP-ban készítettünk el. Ehhez tartozik az adatbázis, amit MySQL-ben hoztunk létre.

A Frontend felel a weboldalunk kinézetéért és a felhasználói felületért. Ezt React vite-ben készítettük el.

Az Asztali alkalmazás (LMS Desktop) felel az adminisztrációs, illetve a könyvtáros felületért. Itt a könyvtárosok tudják kezelni az adatbázisban tárolt adatokat, az adminisztrátorok pedig hozzáférnek a teljes felülethez, ezen felül a felhasználók jogait is kezelni tudják.

A felületekre be lehet jelentkezni a következő minta felhasználókkal:

- admin: adminpassword -> Admin
- johndoe: password123 -> Member
- librarian1: libpass -> Librarian

Felhasználói dokumentáció

Backend

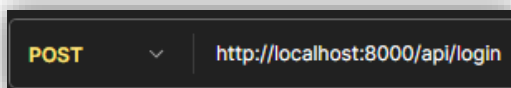
A backend szervert a `start_backend.bat` fájl elindításával lehet futtatni, vagy a „`php -S 127.0.0.1:8000 router.php`” paranccsal. Amennyiben nem elérhető a globális php parancs, akkor „`...\xampp\php\php.exe -S localhost:8000 router.php`”.

Bevezetés:

Az API (Application Programming Interface) különböző végpontokkal rendelkezik. A kérés típusától függően lehet GET, POST, PUT, DELETE. Azonos végpont cím rendelkezhet más-más kérési típussal. A végpontok a következők (`http://localhost:8000/`):

Api/login (POST)

A bejelentkezés POST kérési metódust vár. A végpont megcímzésekor 2 adatot vár a kérési metódus törzsében json formátumban. Az egyik a *username* a másik a *password*. Ezek az adatok alapján eldönti a szoftver, hogy a megadott adatok megegyeznek-e az adatbázisban lévővel. Amennyiben nem vagy rossz formátumban vannak az adatok, akkor különböző hibaüzeneteket írhat ki json formátumban.



ábra 1: végpont és metódus

```
1 {  
2   "username" : "TestUser",  
3   "password" : "notGoodPassword"  
4 }
```

ábra 2: kérés törzse

```
{  
  "error": "wrong username or password;"  
}
```

ábra 3: válaszüzenet

400 Bad Request • 67 ms • 203 B • 🌐

ábra 4: válasz kódja

Helyes adatok megadásával egy token-t küld vissza.

```
{
  "username" : "TestUser",
  "password" : "TestPassword123"
}
```

ábra 5: kérés törzse

200 OK • 98 ms • 374 B • 🌐

ábra 6: válasz kódja

```
{
  "Token": "eyJ0eXAiOiJKV1QiLCJhbGciOiJIUzI1NiJ9.eyJzdWIiOiJUZXRhbnVXNlciIsImh0dHA6Ly9sb2NhbnVNTyE3MyIsIm1hdCI6MTc0NDU3NjYyMSwiZXhwIjoxNzQ0NTgwMjIxLCJ1c2VySUQiOiJlE2fQ.I-iHsqgCzVKZfVQkyQZd-5my-k7iB3z9AY8LigAKDq8"
}
```

ábra 7: válaszüzenet

Api/register (POST)

A regisztrációs végpont új felhasználó létrehozására szolgál. A végpont a kérés törzsében JSON formátumban kapott adatokat vár, és ellenőrzi azok helyességét. A sikeres regisztráció esetén a felhasználó adatai bekerülnek az adatbázisba. Hibás vagy hiányzó adatok esetén JSON formátumban hibaüzenetet küld vissza.

POST http://localhost:8000/api/register

ábra 8: végpont és metódus

```
1 {
2   "error": "Username already exists;"
3 }
```

ábra 9: válaszüzenet

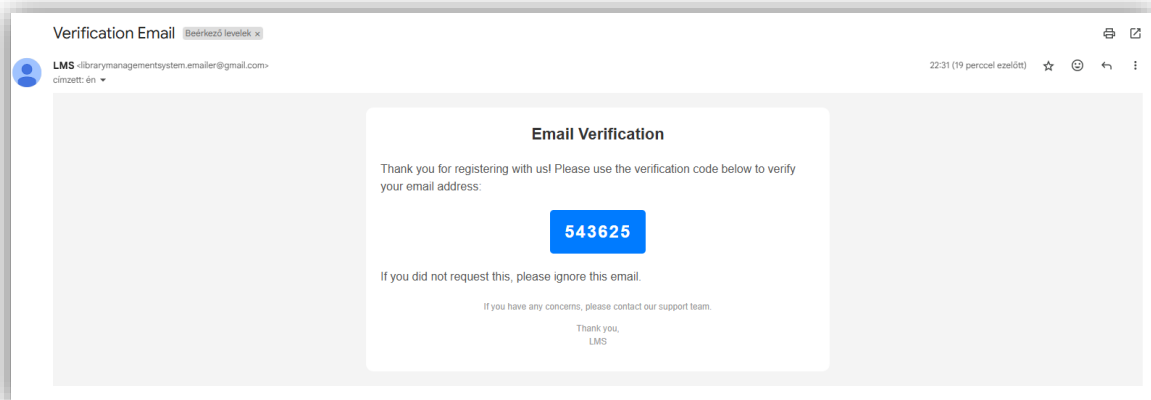
```
1 {
2   "username" : "TestUser",
3   "password" : "TestPassword123",
4   "firstname" : "Test",
5   "lastname" : "User",
6   "email" : "Test.user@gmail.com",
7   "address" : "1 Test St, Testfield",
8   "dateOfBirth" : "1991-01-27"
9 }
```

ábra 10: kérés törzse

400 Bad Request • 91 ms • 200 B • 🌐

ábra 11: válasz kódja, ideje és mérete

Helyes adatok megadásakor a megadott e-mail címre érkezik egy új levél, ami tartalmazza az ellenőrző kódot:



ábra 12: verifikációs e-mail

```
1 {  
2   "username" : "NotExists",  
3   "password" : "TestPassword123",  
4   "firstname" : "Test",  
5   "lastname" : "User",  
6   "email" : "Not.Test.user@gmail.com",  
7   "address" : "1 Test St, Testfield",  
8   "dateOfBirth" : "1991-01-27"  
9 }
```

ábra 13: kérés törzse

```
{  
  "Success": "User created"  
}
```

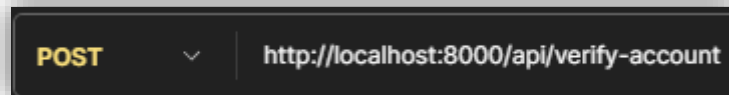
ábra 14: válaszüzenet

```
200 OK • 1.84 s • 181 B • 🌐
```

ábra 15: válasz kódja, ideje és mérete

Api/verify-account (POST)

A verifikációs végpontot akkor használják, amikor egy felhasználó ellenőrzi a regisztrációját egy visszaigazoló kód megadásával. A végpont a kérés törzsében JSON formátumban kapott *verificationCode* adatot vár. Ha a kód helyes, frissíti a felhasználó állapotát az adatbázisban. Ha a kód érvénytelen, hibaüzenetet küld vissza.



ábra 16: végpont és metódus

```
{
  "error": "wrong verify code;"
}
```

ábra 17: válaszüzenet

400 Bad Request • 20 ms • 194 B • 🌐

ábra 18: válasz kódja, ideje és mérete

Helyes adatokkal:

```
{
  "verificationCode" : "617447"
}
```

ábra 19: kérés törzse

```
{
  "Success": "User verified"
}
```

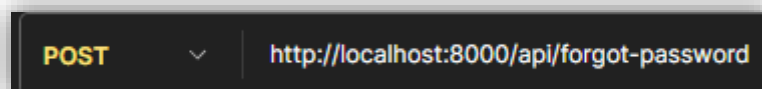
ábra 20: válaszüzenet

200 OK • 39 ms • 182 B • 🌐

ábra 21: válasz kódja, ideje és mérete

Api/forgot-password (POST)

Ez a végpont lehetővé teszi a felhasználó számára, hogy elfelejtett jelszót kérjen. Az e-mail cím alapján egy egyedi token kerül generálásra és elküldésre a felhasználó számára egy visszaállító linken keresztül.



ábra 22: végpont és metódus

```
{
  "email" : "test.com"
}
```

ábra 23: kérés törzse

```
{
  "error": "wrong email format;"
}
```

ábra 24: válaszüzenet

```
400 Bad Request • 8 ms • 195 B • 🌐
```

ábra 25: válasz kódja, ideje és mérete

Helyes adatokkal:

```
{
  "email" : "tester.user@example.com"
}
```

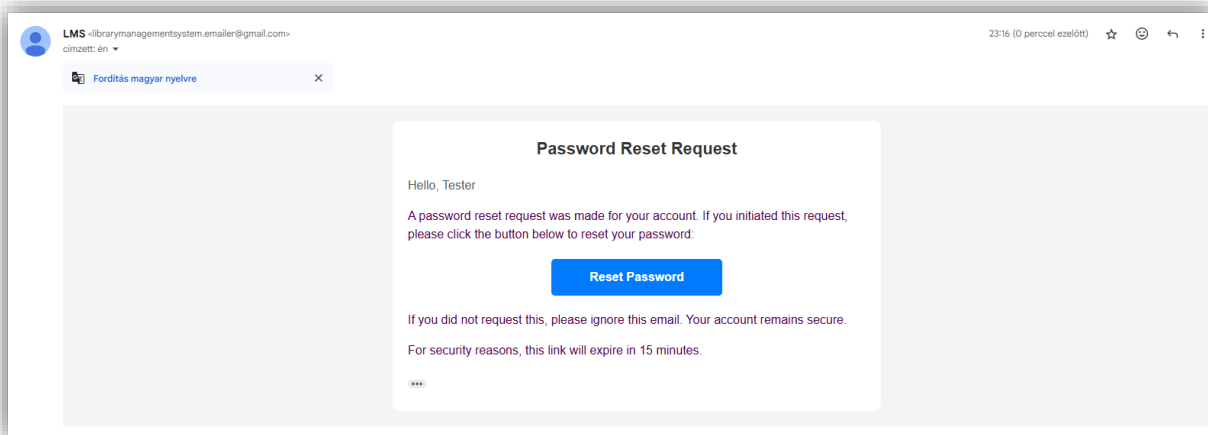
ábra 26: kérés törzse

```
{
  "Success": "Email sent"
}
```

ábra 27: válaszüzenet

```
200 OK • 1.74 s • 179 B • 🌐
```

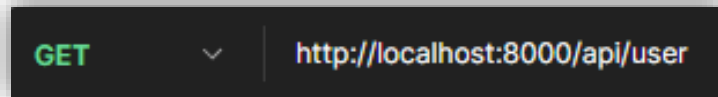
ábra 28: válasz kódja, ideje és mérete



ábra 29: automatikus e-mail

Api/user (GET)

Ennek a végpontnak az eléréséhez szükséges az azonosítás. A bejelentkezéskor kapott token alapján azonosít a backend. Ez a token a header-ben kell, hogy legyen. A felhasználó adatait listázza ki.



ábra 30: végpont és metódus

```
[
  {
    "email": "tester.user@example.com",
    "username": "TestUser",
    "firstname": "Test",
    "lastname": "User",
    "address": "1 Test St, Testfield",
    "DateOfBirth": "1991-01-27",
    "Role": "Member"
  }
]
```

ábra 31: válaszüzenet

200 OK • 19 ms • 327 B • 🌐

ábra 32: válasz kódja, ideje és mérete

Api/user (PUT)

A tokennel való azonosítás után a végpont megvizsgálja az adatokat. Ami üres tehát nem módosult azt az adatot nem viszi fel az adatbázisba. A felhasználó a jelszavát is módosíthatja, de csak akkor, ha megadta a jelenlegi jelszavát, viszont az e-mail címét nem tudja módosítani. A felhasználó csak olyan adatokat adhat meg amiket megenged a validáció amennyiben ez a feltétel nem teljesül hibaüzenet kerül elküldésre.



ábra 33: végpont és metódus

```
{
  "username": ".?",
  "firstname": "Test",
  "lastname": "User",
  "address": "1 Test St, Testfield",
  "DateOfBirth": "1991-01-27"
}
```

ábra 34: kérés törzse

```
{
  "error": "invalid username;"
}
```

ábra 35: válaszüzenet

400 Bad Request • 46 ms • 193 B • 🌐

ábra 36: válasz kódja, ideje és mérete

Helyes adatokkal:

```
{
  "username": "TestUserNew",
  "firstname": "Test",
  "lastname": "User",
  "address": "1 Test St, Testfield",
  "DateOfBirth": "1991-01-27"
}
```

ábra 37: kérés törzse

```
{
  "Success": "User updated"
}
```

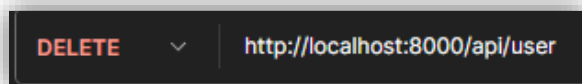
ábra 38: válaszüzenet

200 OK • 49 ms • 181 B • 🌐

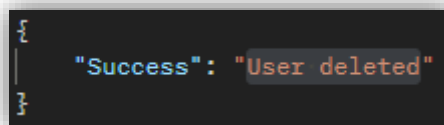
ábra 39: válasz kódja, ideje és mérete

Api/user (DELETE)

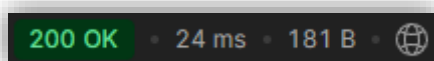
A felhasználói token dekódolva az ID alapján kitörli az adott felhasználót. A token ha már nem létező ID-val rendelkező felhasználóra mutat akkor nem fut le a az adatbázis módosítás. Ilyenkor egy hibaüzenet látható.



ábra 40: végpont és metódus

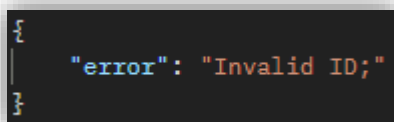


ábra 41: válaszüzenet

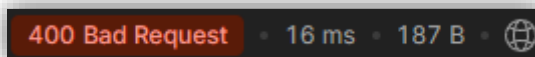


ábra 42: válasz kódja, ideje és mérete

A kérés futtatása újra a törlés után a következő eredményre jut:



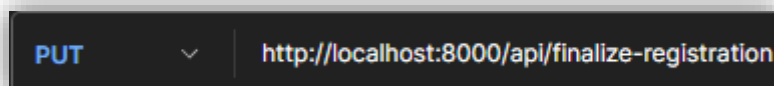
ábra 43: válaszüzenet



ábra 44: válasz kódja, ideje és mérete

Api/finalize-registration (PUT)

Ez a végpont csak azoknak a felhasználóknak elérhető, akik nem regisztráltak a weboldalon, de már regisztráltak a könyvtárban. Az kérés törzsében kell *email*, *username* és *password* azonosító. Ezek hiányában hibaüzenet várható. A password a könyvtáros által felvitt felhasználónak a generált jelszava. A felhasználónév és jelszó megadásával bejelentkezik a felhasználó, de csak akkor, ha már nem regisztrált az oldalon, ha a bejelentkezés sikeres akkor a felhasználónak módosul az e-mail címe. A sikeres regisztrációt egy e-mail is jelzi, ami már a regisztrációnál is megjelent, tehát az ellenőrző levél.



ábra 45: végpont és metódus

```
{
  "email" : "tester@example.com",
  "password" : "TestPassword123"
}
```

ábra 46: kérés törzse

```
{
  "error": "Missing values;"
}
```

ábra 47: válaszüzenet

400 Bad Request • 9 ms • 191 B • 🌐

ábra 48: válasz kódja, ideje és mérete

Helyes adatokkal:

```
{
  "email" : "tester@example.com",
  "username" : "TestUserNew",
  "password" : "TestPassword123"
}
```

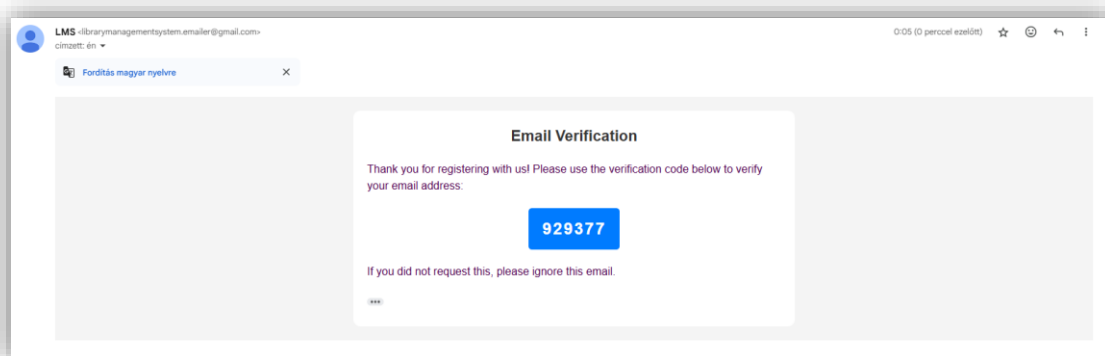
ábra 49: kérés törzse

```
{
  "Success": "Email sent"
}
```

ábra 50: válaszüzenet

200 OK • 2.98 s • 179 B • 🌐

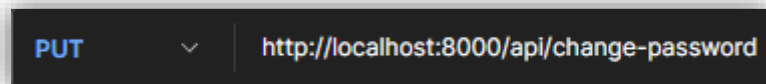
ábra 51: válasz kódja, ideje és mérete



ábra 52: verifikációs e-mail

Api/change-password (PUT)

A végpont csak tokennel elérhető. Ez a kérés várhatóan csak akkor fog megtörténni, ha a felhasználó kérte, hogy módosíthassa a jelszavát. A várt adatok a *password* és a *passwordAgain*. Természetesen az új jelszónak meg kell felelnie az elvárt szabályoknak, ha ez nem történik meg akkor hibaüzenetbe fut a felhasználó.



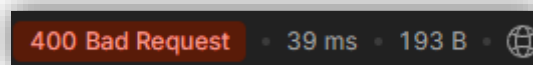
ábra 53: végpont és metódus

```
{  
  "password" : "notvalid",  
  "passwordAgain" : "notvalid"  
}
```

ábra 54: kérés törzse

```
{  
  "error": "invalid password;"  
}
```

ábra 55: válaszüzenet



ábra 56: válasz kódja, ideje és mérete

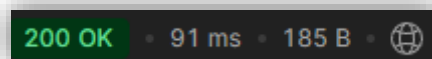
Helyes adatokkal:

```
{  
  "password" : "TestPassword1234",  
  "passwordAgain" : "TestPassword1234"  
}
```

ábra 57: kérés törzse

```
{  
  "Success": "Password updated"  
}
```

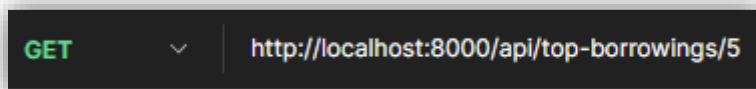
ábra 58: válaszüzenet



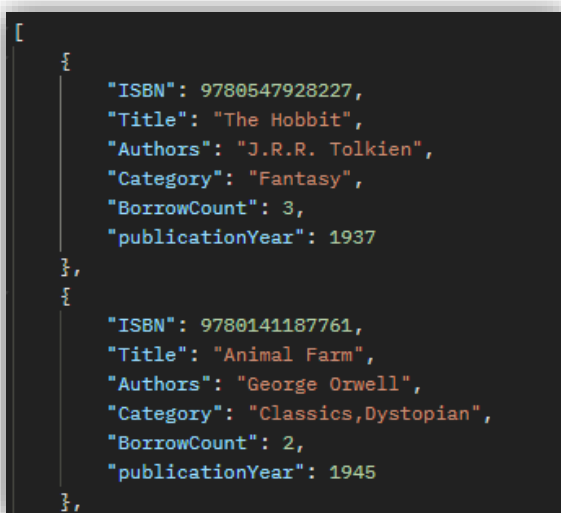
ábra 59: válasz kódja, ideje és mérete

Api/top-borrowings/{limit} (GET)

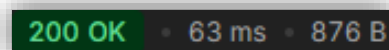
A végpont megcímzésekor szükséges egy számot megadni. A szám azt jelenti, hogy a lekérdezésben ez a szám fog szerepelni, mint limit. A lekérdezés, ha jó akkor a könyv(ek) adataival fog visszatérni, ami ISBN, Title, Authors, Category, BorrowCount, publicationYear. A BorrowCount az az érték ahányszor kikölcsönözték a könyvet. A megjelenítés legtöbbször kölcsönzött könyvvvel kezdi a json-t.



ábra 60: végpont és metódus



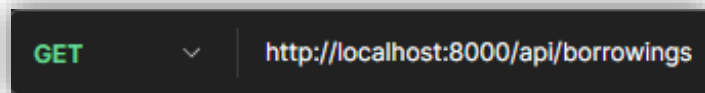
ábra 61: válaszüzenet



ábra 62: válasz kódja, ideje és mérete

Api/borrowings (GET)

A végpont elérésekor kilistázza a felhasználóhoz tartozó összes kölcsönzést. Az azonosítás tokennel történik. A könyv(ek) adatait és a kölcsönzéssel kapcsolatos információkat írja ki.



ábra 63: végpont és metódus

```
[
  {
    "ISBN": 9780141187761,
    "Publisher": "Penguin Random House",
    "Title": "Animal Farm",
    "Authors": "George Orwell",
    "Category": "Classics,Dystopian",
    "publicationYear": 1945,
    "BorrowDate": "2024-10-22",
    "DueDate": "2024-11-05",
    "ReturnDate": null
  },
  {
    "ISBN": 9780743273565,
    "Publisher": "HarperCollins",
    "Title": "The Great Gatsby",
    "Authors": "F. Scott Fitzgerald",
    "Category": "Gothic Fiction",
    "publicationYear": 1925,
    "BorrowDate": "2024-10-08",
    "DueDate": "2024-10-22",
    "ReturnDate": null
  },
]
```

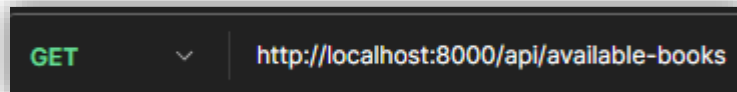
ábra 64: válaszüzenet

200 OK • 31 ms • 1.05 KB • 

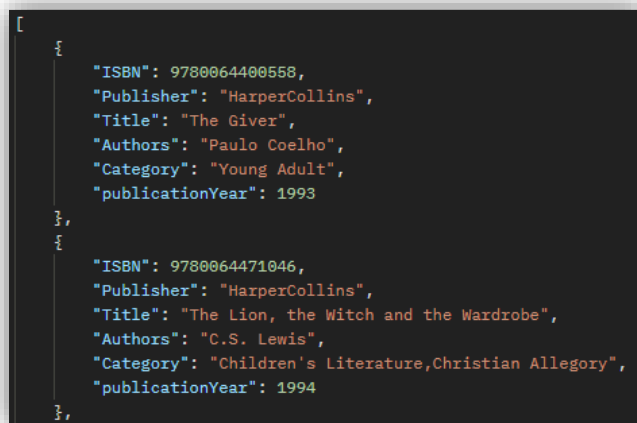
ábra 65: válasz kódja, ideje és mérete

Api/available-books (GET)

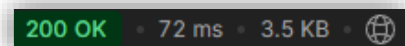
A végpont eléréséhez nem szükséges az azonosítás. Kilistázza az összes elérhető könyvet és azok adatait. Egy könyv akkor nem elérhető, ha valaki kikölcsönözte vagy lefoglalta.



ábra 66: végpont és metódus



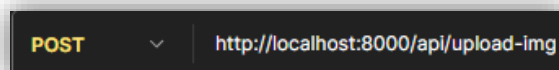
ábra 67: válaszüzenet



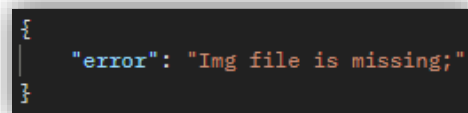
ábra 68: válasz kódja, ideje és mérete

Api/upload-img (POST)

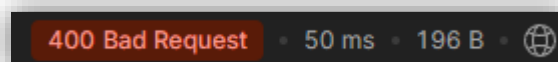
A kép feltöltéséhez a /api/upload-img végpontot kell megcímezni. megvizsgálásra kerül, hogy van-e a kérésben kép, amennyiben igen akkor a képet további ellenőrzések után feltölti a *booksImages* mappába. A kép nevének az ISBN-nek kell lennie és a kiterjesztés CSAK jpg vagy png vagy jpeg lehet. A backend nem dolgozik fel más formátumot.



ábra 69: végpont és metódus

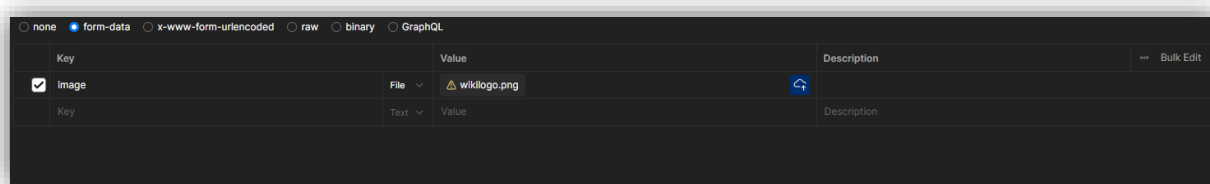


ábra 70: válaszüzenet



ábra 71: válasz kódja, ideje és mérete

Helyesen feltöltött képpel:



ábra 72: kérés törzse

```
{
  "Success": "Image uploaded"
}
```

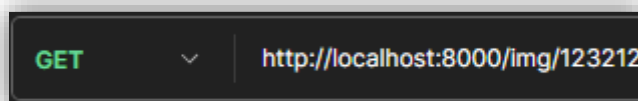
ábra 73: válaszüzenet

200 OK • 31 ms • 163 B • 🌐

ábra 74: válasz kódja, ideje és mérete

img/{ISBN} (GET)

A végponthoz nem kell azonosítás, mivel ezeket az adatokat bárki elérheti. A várt adat egy az url-be beírt *ISBN* szám, ami a könyvhöz tartozó képet tölti be, ha a könyv nem található a *booksImages* mappában, akkor a *missing.png* fájlt tölti be alapértelmezetten. Abban az esetben, ha az ISBN nem megfelelő formátumban kerül átadásra egy hibakód kerül átadásra. A fájl formátumát nem kell megadni, mert a végpont automatikusan lekéri a 3 opció közül az összeset, és ha talált egy fájlt akkor azt tölti be.



ábra 75: végpont és metódus

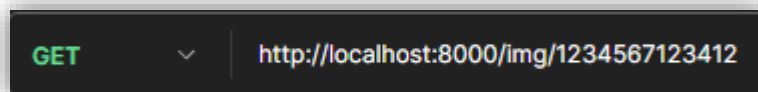
```
{
  "error": "Invalid ISBN;"
}
```

ábra 76: válaszüzenet

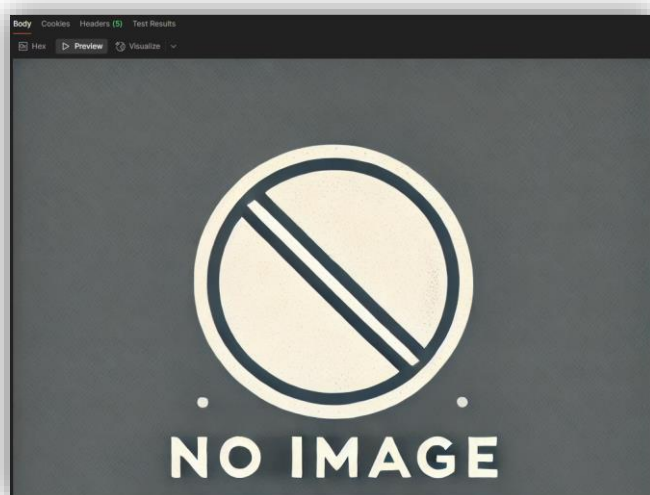
400 Bad Request • 19 ms • 189 B • 🌐

ábra 77: válasz kódja, ideje és mérete

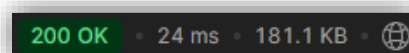
Nem található kép:



ábra 78: végpont és metódus

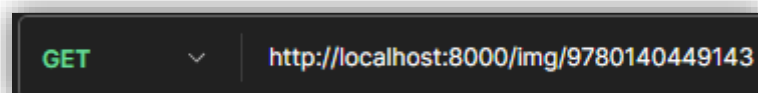


ábra 79: válaszüzenet (kép formátumban)



ábra 80: válasz kódja, ideje és mérete

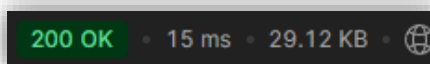
Helyes adatokkal:



ábra 81: végpont és metódus



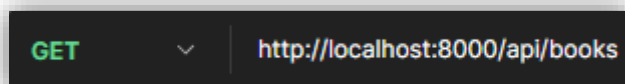
ábra 82: válaszüzenet (kép formátumban)



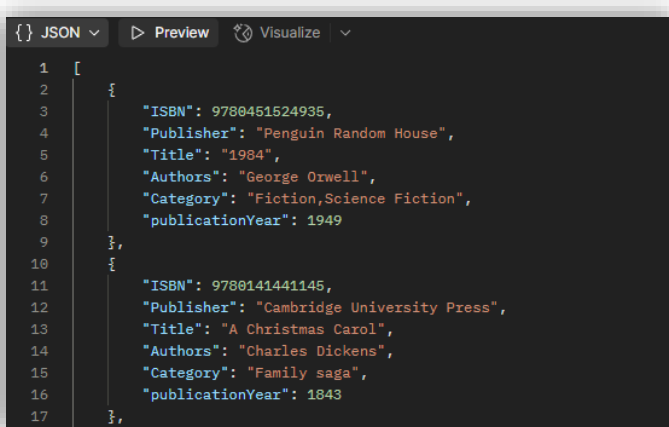
ábra 83: válasz kódja, ideje és mérete

Api/books (GET)

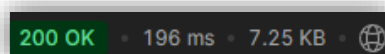
Az `api/books` végpont a könyveket listázza ki. Az url-be több adat is megadható például `api/books/category/fiction`, így az összes olyan könyv kerül kilistázásra, amelynek a kategóriája *fiction*. A végpont elérésekor semmi adatot megadva az összes könyvet és adatait küldi vissza json formátumban.



ábra 84: végpont és metódus

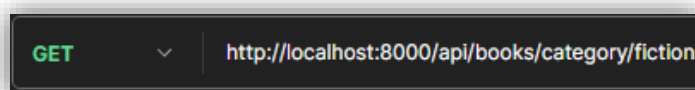


ábra 85: válaszüzenet

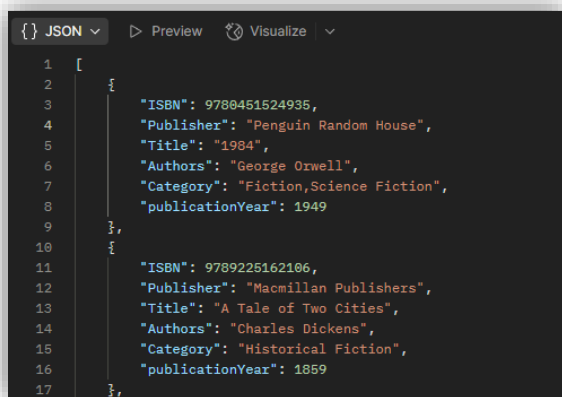


ábra 86: válasz kódja, ideje és mérete

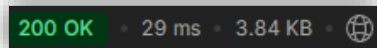
Adatok megadásával:



ábra 87: végpont és metódus



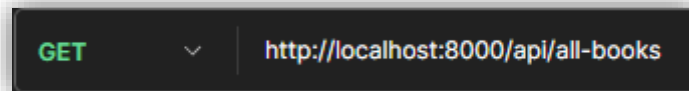
ábra 88: válaszüzenet



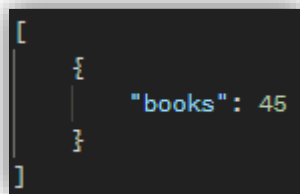
ábra 89: válasz kódja, ideje és mérete

Api/all-(books, users, borrowings) (GET)

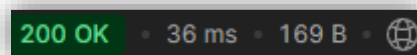
A fenti végpontok elérése azonos feladat elvégzésére szolgál, csak más-más táblában valósul meg. A feladata, hogy egy értékkel térjen vissza, ami megszámlolt adatot tartalmazza például a `/api/all-books` elérésekor azt számolja meg, hogy mennyi a jelenlegi könyveknek a száma.



ábra 90: végpont és metódus



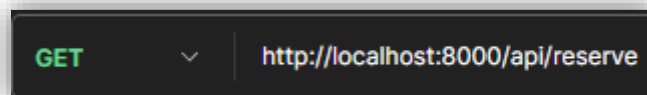
ábra 91: válaszüzenet



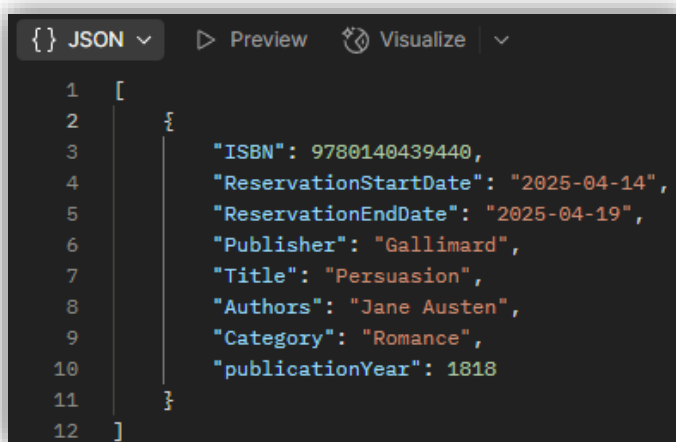
ábra 92: válasz kódja, ideje és mérete

Api/reserve (GET)

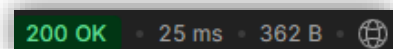
A végpont elérésekor azonosítani kell a felhasználót egy tokennel. A megadott token alapján lekérdezésre kerül az összes olyan könyv, amit az adott felhasználó foglalt le. Az összes lefoglalás nem haladhatja meg a 3-at.



ábra 93: végpont és metódus



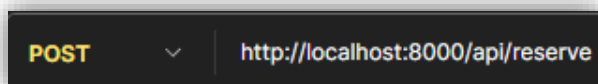
ábra 94: válaszüzenet



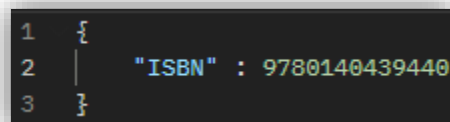
ábra 95: válasz kódja, ideje és mérete

Api/reserve (POST)

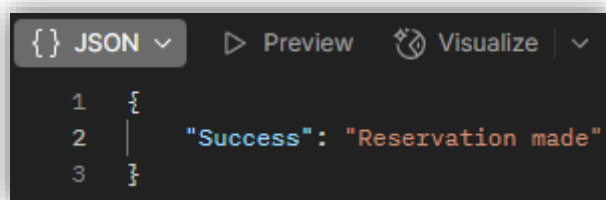
A végpont megcímzésekor szükséges elküldeni a kérés törzsében az ISBN-t. A végpont token használ az azonosításhoz, anélkül nem is lehet elérni a végpontot. Az eléréskor a backend megvizsgálja az adatokat, ha az adatok nem jók az alapján hibaüzenetet küld vissza. A helyes adatok megadásával az adatbázisban lefoglalja a felhasználó részére a könyvet. A könyv, addig amíg le nem jár a foglalás nem lesz látható az elérhető könyvek között.



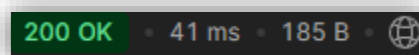
ábra 96: végpont és metódus



ábra 97: kérés törzse



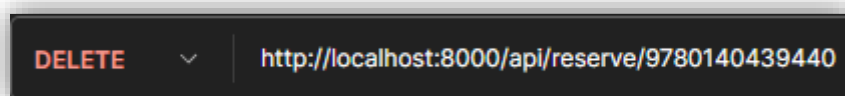
ábra 98: válaszüzenet



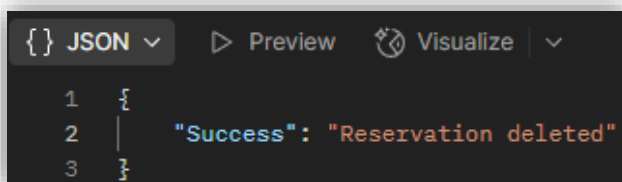
ábra 99: válasz kódja, ideje és mérete

Api/reserve/{ISBN} (DELETE)

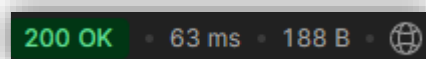
Az api/reserve/{ISBN} végpont eléréséhez azonosítás szükséges tokennel. Az ISBN-t az url-be kell beírni, aminek meg kell felelnie a feltételeknek. Az ISBN-nek helyes formátumúnak kell lennie és a felhasználóhoz kell tartoznia, ha ez nem teljesül hibaüzenet kerül visszaküldésre a backendtől.



ábra 100: végpont és metódus



ábra 101: válaszüzenet



ábra 102: válasz kódja, ideje és mérete

Tesztelés

A mellékelt Postman gyűjtemény a Library Management System (LMS) API tesztelésére szolgál, és tartalmazza a könyvekkel kapcsolatos végpontokhoz tartozó HTTP kéréseket. A hozzáférés token alapú hitelesítést használ, amelyet a bejelentkezés során kapott JWT token biztosít. A Postman gyűjtemény a *tests* mappában található és a hozzá tartozó tesztek is. A teszteléshez a végpontokhoz tartozó json fájlt kell betölteni és futtatni.

Frontend

Weboldal célja:

Az **LMS** (*Library Management System*) nevű webes könyvtári kölcsönzéseik megtekintésére és könyvek lefoglalására nyújt lehetőséget a felhasználók számára.

Rendszerkövetelmények:

- Modern webböngészők: Google Chrome, Mozilla Firefox, Brave, Opera, Microsoft Edge, Safari
- Képernyőfelbontás: reszponzív, alkalmazkodik a kijelző felbontásához

Weboldal futtatása:

A frontend webszervert futtatásának első lépése az *npm install*, a szükséges node module-ok telepítéséhez, majd ezt követően a *%/LMS/Website/React* mappában kiadni az *npm run dev* parancsot vagy mindezek helyett használható *%/LMS/Website/* mappában található *set_up_and_start_website.bat* fájl a weboldal futtatáshoz.

Kezdőlap (Home)

A kezdőlap a weboldal belépési pontja, ahol a felhasználók áttekinthetik:

- Az adatbázisban lévő könyvek számát,
- A regisztrált felhasználók számát,
- Az összes kölcsönzés számát,
- Az 5 legtovább kikölcsönzött könyv,
- Innen érhetők el a fejlécen található bejelentkezési, regisztrációs és könyvböngészési oldalak.

A statisztikák nem valós időben frissülnek, azaz cache elérés van.

Felhasználói hitelesítés

A weboldal biztosítja a felhasználók biztonságos hitelesítését és hozzáférését.

- Bejelentkezés (Login): A regisztrált felhasználók bejelentkezhetnek a fiókjukba.
- Regisztráció (Register): Új felhasználók regisztrálhatnak a rendszerbe.
 - Követelmények:
 - Vezetéknév és Keresztnév (Firstname and Lastname): Az első betűnek nagy-nak kell lennie.

- Email: Email cím formátum.
- Felhasználónév (Username): Egyedi, nem egyezhet más felhasználónevével.
- Jelszó (Password): Minimum 8 karakter, tartalmaznia kell számokat és betűket.
- Születési dátum (Date of Birth):
- Cím (Address): Házzám + Közterület neve + Közterület jellege + , + Város
- Regisztráció véglegesítése (Finalize Registration): Előregisztrált felhasználók véglegesítése.
- Email megerősítés (Verify): A két fajta regisztráció során a felhasználók email címének megerősítése szükséges.
- Jelszókezelés:
 - Elfelejtett jelszó (Forgot Password): Lehetővé teszi a felhasználók számára, hogy új jelszót kérjenek emailben.
 - A link 15 percig érvényes.
 - Jelszó módosítása (Change Password): Az emailben kapott linken keresztül a felhasználó megváltoztathatja a jelszavát.
 - A profil oldalon vagy az emailben kapott linken keresztül lehetséges.

Könyvek böngészése és szűrése (Books)

- A felhasználók böngészhetik az elérhető könyvek listáját, amely szűrhető különböző kategóriák alapján:
 - Szerző
 - Kiadó
 - Kiadási év
 - Kategória
- A keresősáv segítségével a felhasználók könyvcím vagy szerző alapján kereshetnek könyvekre.
- A találatok kártyák formájában jelennek meg, amelyek tartalmazzák a könyvcímét, szerzőjét és kategóriáját, ezen felül egy *Reserve* gombot, amelyre rákattintva a felhasználó letudja foglalni a könyvet.
 - A foglalás 7 napig érvényes.
 - Foglalás meghosszabbítása esetén a könyvtárost kell felkeresni.

Felhasználói irányítópult (Dashboard)

A bejelentkezett felhasználók számára elérhető az irányítópult, amely a következő funkciókat tartalmazza.

- Kölcsönzések (Borrowings): A felhasználó által kölcsönzött könyvek listája, visszahozatali határidővel.
- Foglalások (Reservations): A felhasználó által lefoglalt könyvek listája.
 - Lemondás: *Cancel* (Mégse) gomra kattintva.
- Profil (Profile): A felhasználó személyes adatai.

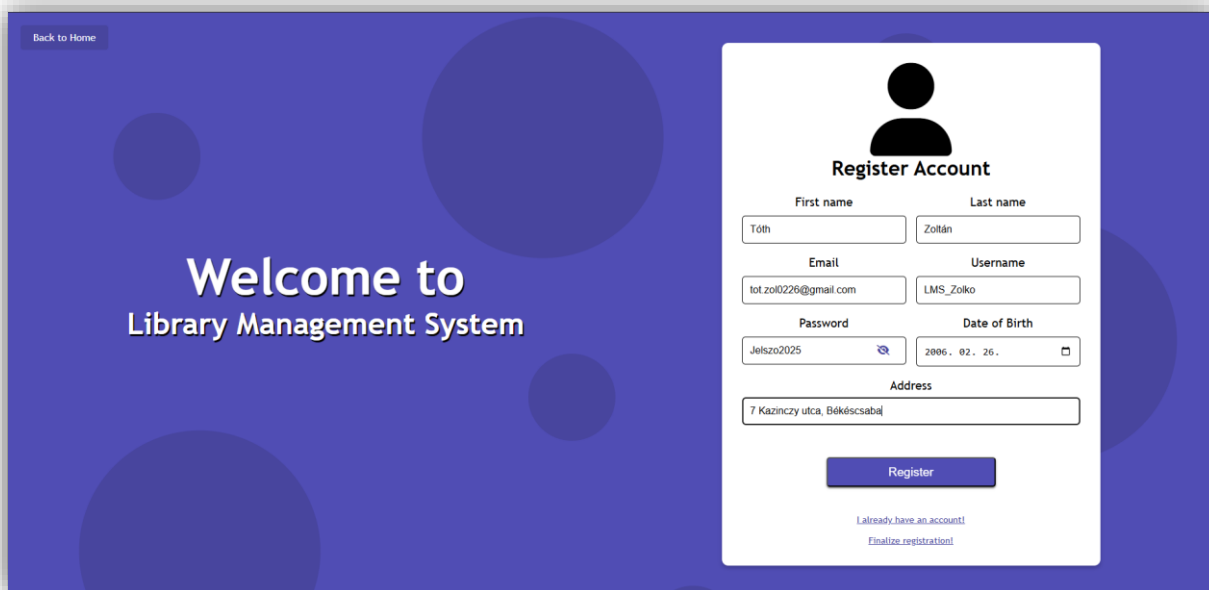
Az irányítópult csak a bejelentkezett felhasználók számára érhető el, és a hozzáférést a *PrivateRoute* komponens biztosítja.

Hibakezelés

- 404-es hibaoldal (NotFound): Ha a felhasználó olyan oldalra navigál, amely nem létezik, egy 404-es hibaoldal jelenik meg.

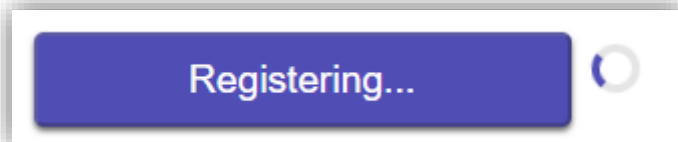
Regisztráció folyamata:

A főoldalon a felhasználó a *Register* gombra kattintva egy űrlapot talál, ahova saját adatait megadva tud regisztrálni. Az űrlap kitöltése után egy email verifikációs oldalon fogja találni magát a felhasználó, ahova az email-jévhez kapott kóddal tud aktiválni.

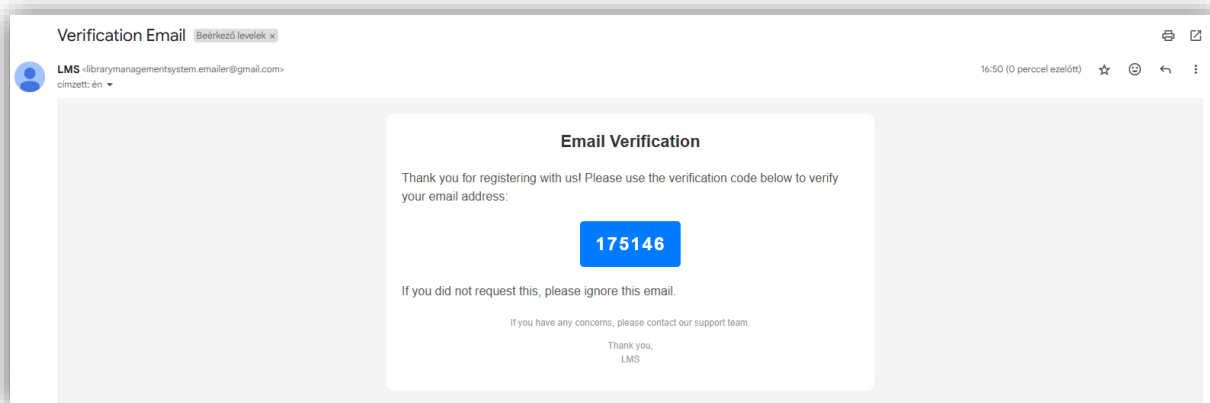


The screenshot shows a web application interface with a dark blue background and large, faint circles. On the left, a 'Back to Home' button is visible. The main content area features a 'Welcome to Library Management System' message. On the right, there is a 'Register Account' form. The form includes a user icon, a title 'Register Account', and several input fields: 'First name' (containing 'Tóth'), 'Last name' (containing 'Zoltán'), 'Email' (containing 'tot.zoltan22@gmail.com'), 'Username' (containing 'LMS_Zolto'), 'Password' (containing 'Jelszo2025'), 'Date of Birth' (containing '2006. 02. 26.'), and 'Address' (containing '7 Kazinczy utca, Békéscsaba'). A 'Register' button is at the bottom of the form. Below the button, there are two links: 'I already have an account!' and 'Finalize registration!'.

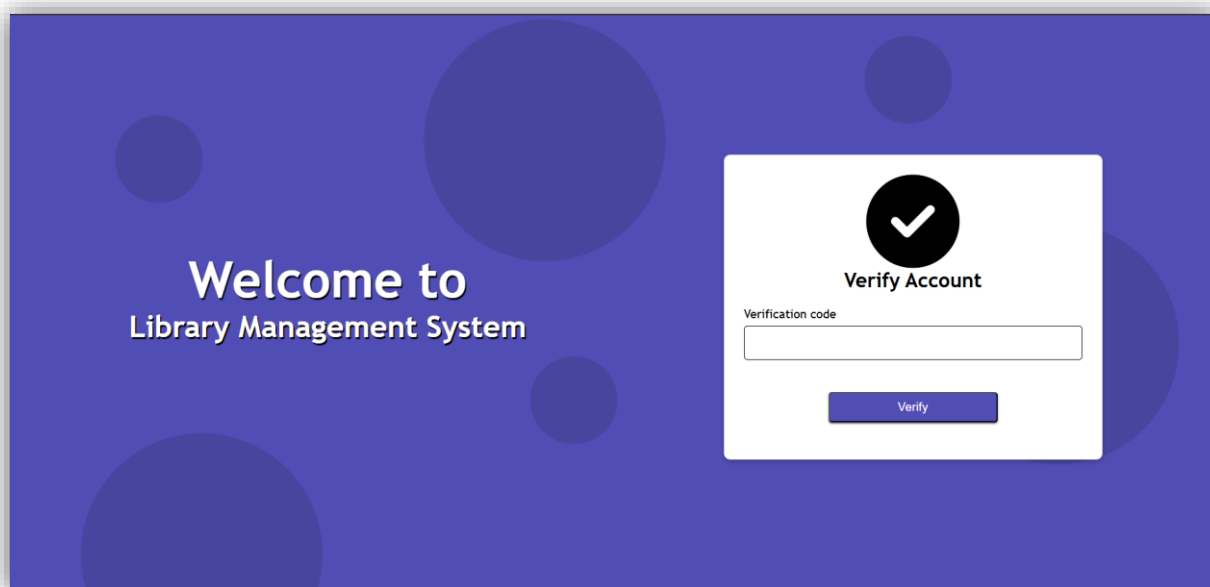
ábra 103: Regisztráció



ábra 104: Töltés közbeni animáció



ábra 105: Verifikációs kód



ábra 106: Fiók visszaigazolása

A sikeres regisztráció után átirányít a *Login* oldalra, ahol a felhasználó be tud jelentkezni a fiókjába.

Back to Home

Library Management System

Login

Username
LMS_Zolko

Password
***** Show

[Forgot your password?](#)

Login

[Create new account?](#)

ábra 107: Bejelentkező

Bejelentkezés után a felhasználó az irányítópulton találja magát, ahol kölcsönzéseit, lefoglalt könyveit és a profil adatait tudja megtekinteni.

LMS

Hi, LMS_Zolko

Logout

Borrowings Reservations Profile

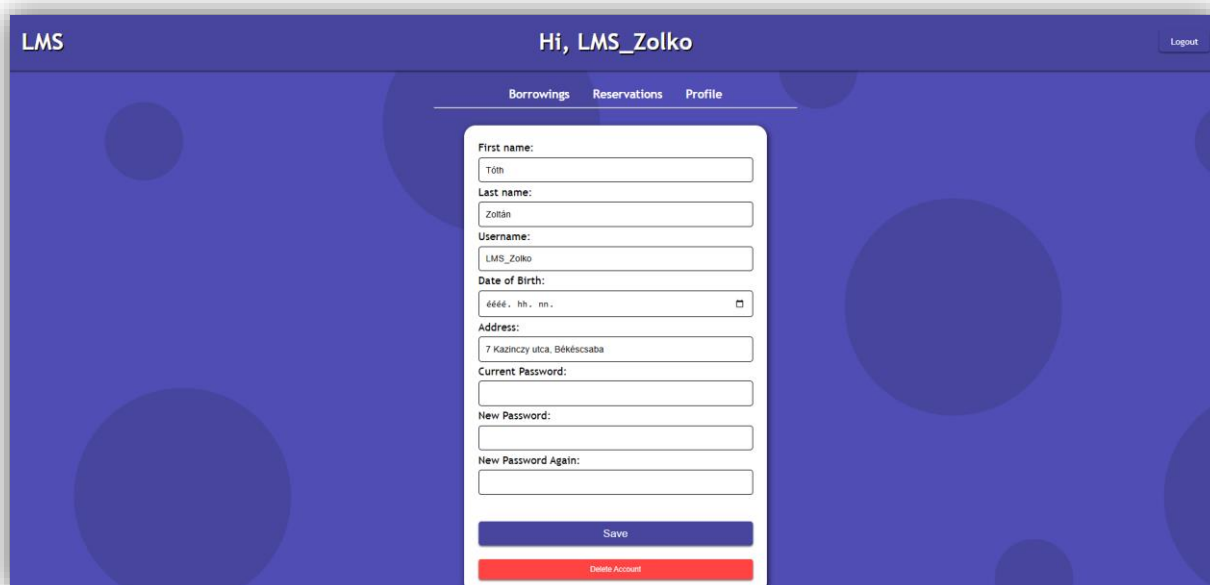
ISBN	Title	Author	Borrowings Date	Due Date
9780064400558	The Giver	Paulo Coelho	2025-05-02	2025-06-02

ábra 108: Kölcsönzések



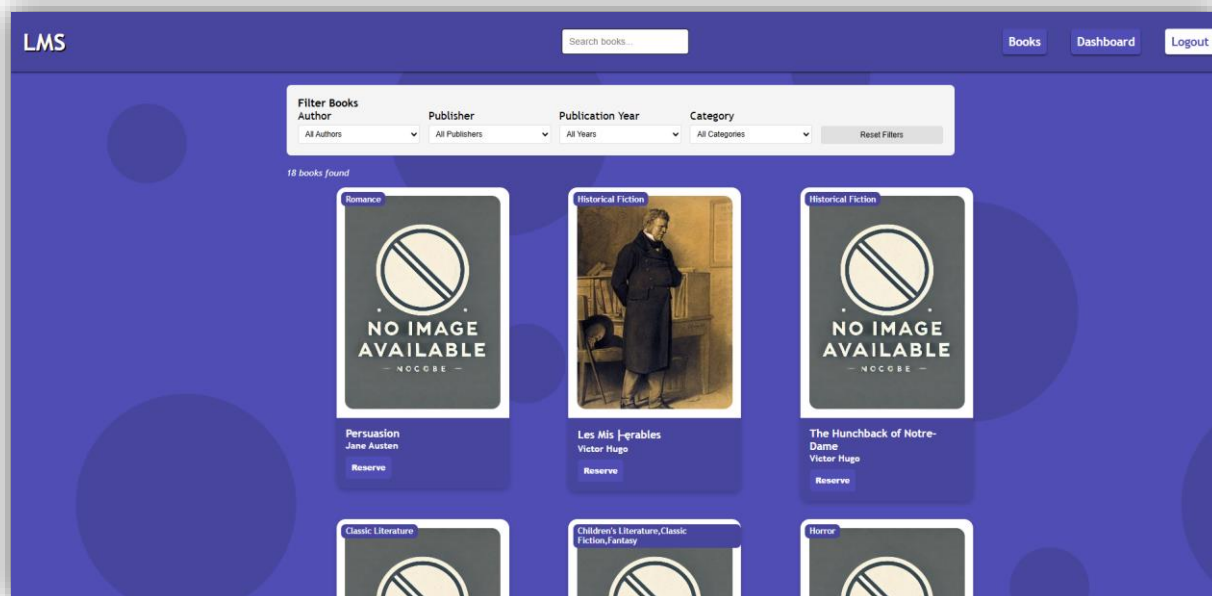
ábra 109: Foglalások

A felhasználó a profilját mind frissíteni, mind törölni is tudja.



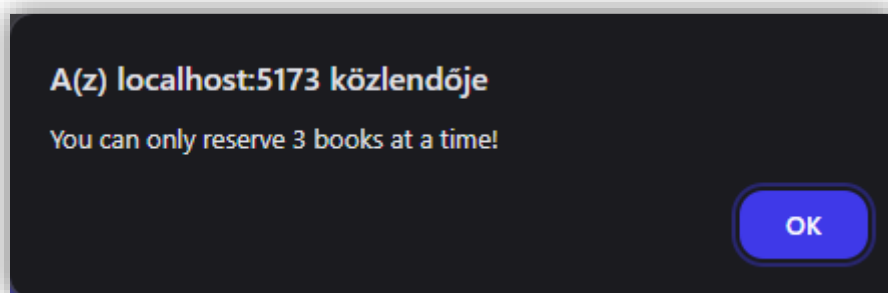
ábra 110: Fiók frissítése

A bejelentkezett felhasználó a /books oldalon tud lefoglalni könyvet. Maximum 3 a megengedett könyv lefoglalás.



ábra 111: Könyvek

Ha több könyvet szeretne kikölcsönözni, ez az alert box-ot kapja:



ábra 112: Foglálási limit

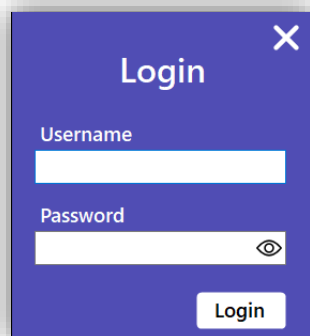
Asztali alkalmazás (LMS Desktop)

Az asztali alkalmazás az adminisztrátorok és a könyvtárosok felülete, amely lehetővé teszi az adatbázis statisztikáinak megjelenítését, illetve a könyvek, kölcsönzések, foglalások, kategóriák, írók, kiadók és felhasználók megtekintését, hozzáadását, frissítését és törlését.

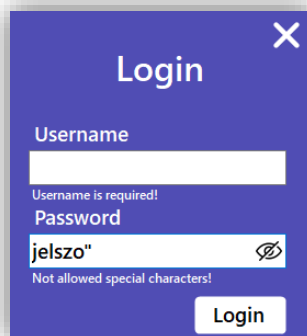
Az alkalmazás telepíthető a .../LMS Desktop/LMS Desktop Installer.exe-vel. A működéshez szükséges az adatbázis, amit a Database mappában található Database.sql fájl tartalmaz minta adatokkal. A kép feltöltéséhez szükséges futtatni a .../Website/start_backend.bat fájlt.

Bejelentkezési felület (Login page)

A bejelentkezéshez szükséges egy felhasználónév, ami nem tartalmazhatja a tiltott karaktereket: \ és ". (Például a következő felhasználónevek érvénytelenek: user\name, "admin". A jelszó megadása szintén kötelező, ami nem tartalmazhatja az előbb felsorolt tiltott karaktereket. A rendszer kizárólag adminisztrátoroknak vagy könyvtárosoknak engedélyezi a bejelentkezést, ha sima felhasználóként próbálunk bejelentkezni, erre figyelmeztet a rendszer. A jobb felső sarokban lévő X gombra kattintva be lehet zárni a programot. A jelszó mezőben található szem ikonra kattintva megtekinthetjük a jelenleg beírt jelszót. Végül egy Login gomb, amivel végrehajthatjuk az ellenőrzést és beléphetünk a rendszerbe. A jelszó írása közben valós időben láthatjuk a hibákat a felhasználónévvel vagy jelszóval amint azt a 2. ábra is szemlélteti.



ábra 113: Bejelentkező felület



ábra 114: Jelszókezelés

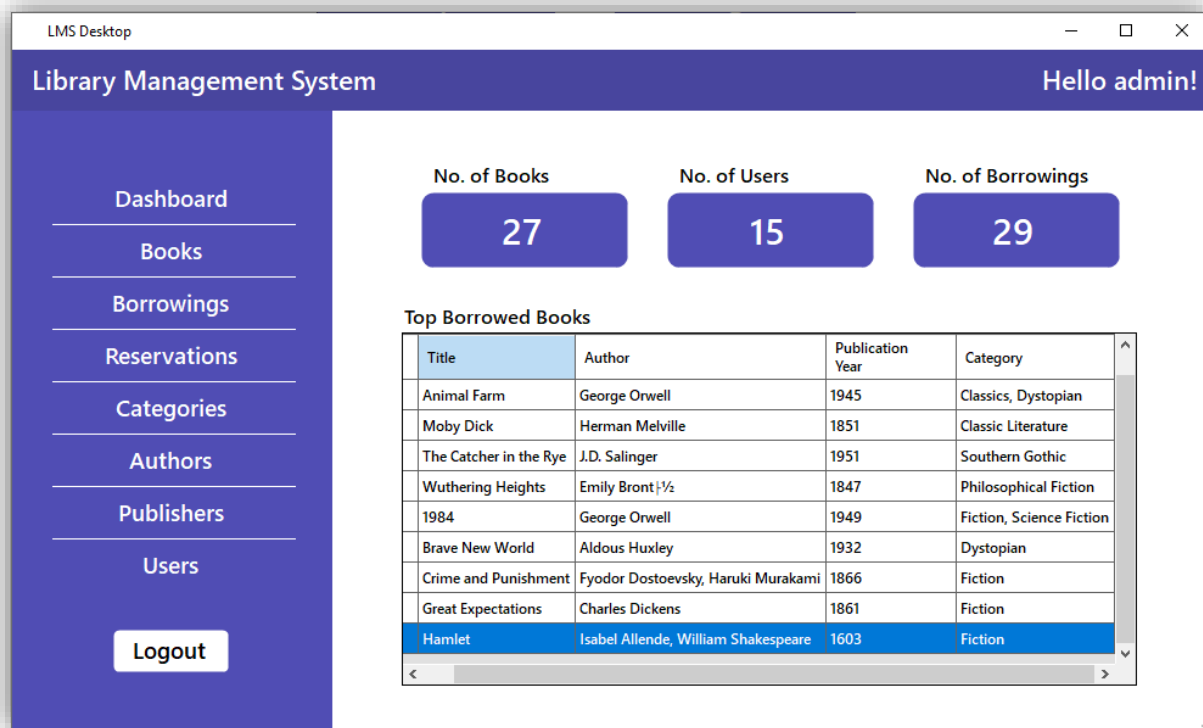
Műszerfal (Dashboard)

A műszerfal felel az adatbázisban tárolt adatok statisztikáinak kiírására, azaz:

- A jelenleg nyilván tartott könyvek száma
- Az összes felhasználó létszáma
- Minden eddigi kölcsönzés száma
- És a 10 legnépszerűbb könyv kölcsönzések száma alapján – Itt kilistázásra kerül az adott könyv kölcsönzéseinek száma, a könyv címe, szerzője, a kiadás éve és a könyv kategóriája.

A bal alsó sarokban lévő „**kijelentkezés**” gomb visszadob a bejelentkezés oldalra, ahol újra szükséges a bejelentkezés. Ezenfelül a jobb felső sarokban szerkeszthetőek a **bejelentkezett felhasználó** adatai (4. ábra).

Ezen a felületen a „**jelszó változtatása**” opciót kiválasztva a felhasználónak lehetősége van megváltoztatni a jelszavát, ehhez valós időben megjelenik, hogy mire van még szükség a megfelelő jelszó kiválasztásához (5. ábra).



ábra 115: Műszerfal

Profile Settings

First name: Admin

Last name: User

Username: admin

Date of Birth: 08/ 11/ 1975

Email: admin@example.com

Address: 1 Admin Plaza, Springfield

Change Password...

Save Cancel

ábra 116: Profilbeállítások

Change Password

Current Password: [Field]

New Password: [Field]

At least one...

- × 8-16 characters
- × Upper case letter
- × Lower case letter
- × Special character or number

Password Again: [Field]

✓ Passwords match

Save Cancel

ábra 117: Jelszóváltoztatás

Könyvek (Books)

A „könyvek” oldalon tudjuk kezelni az adatbázisban tárolt könyveket. Az oldal első gombja egy **frissítés** gomb, melynek megnyomásával az adatbázisból lekérdezésre kerülnek a könyvek ezzel frissítve a táblázatot.

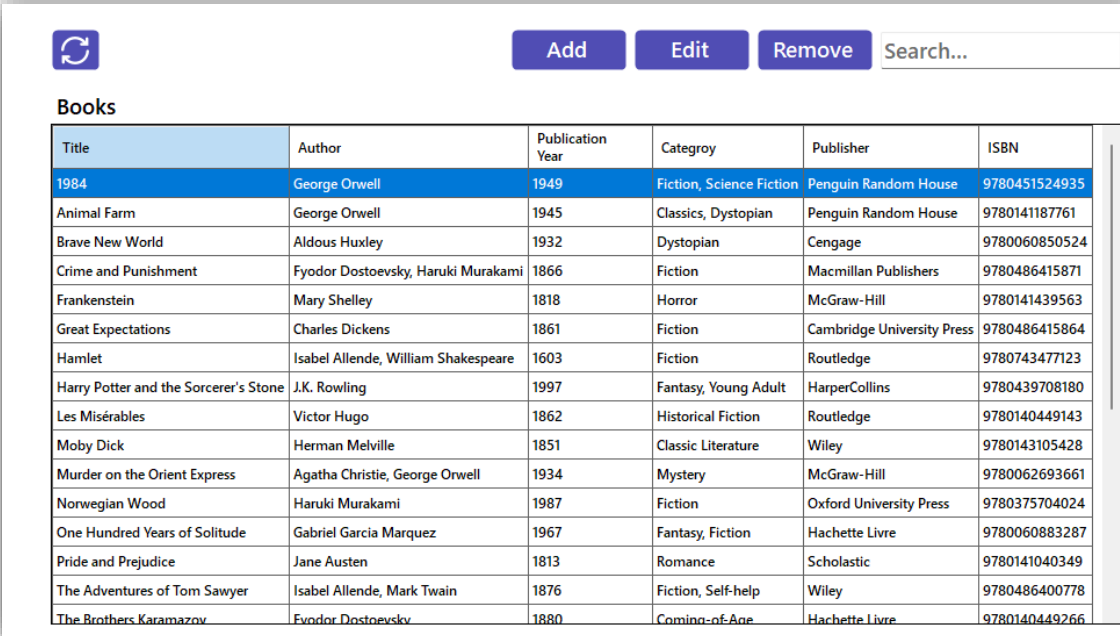
Az ezt követő gombbal megnyílik egy ablak (7. ábra), ahol új könyvet tudunk **hozzáadni**. Itt van lehetőségünk opcionálisan egy kép feltöltésére a könyvről, ami maximum 5 MB lehet és PNG, JPG vagy JPEG formátumú.

Majd következik a „**módosítás**” gomb, mely megnyit egy ablakot (8. ábra), betölti a kiválasztott könyv adatait és módosíthatjuk a kiválasztott könyvet. Itt szintén fel lehet tölteni egy képet a könyvről vagy módosítani a régít az előző pontban leírt követelményekkel.

Az „**eltávolítás**” gombbal pedig akár több könyvet kiválasztva is törölhetjük ezeket az adatbázisból egy megerősítést követően (9. ábra).

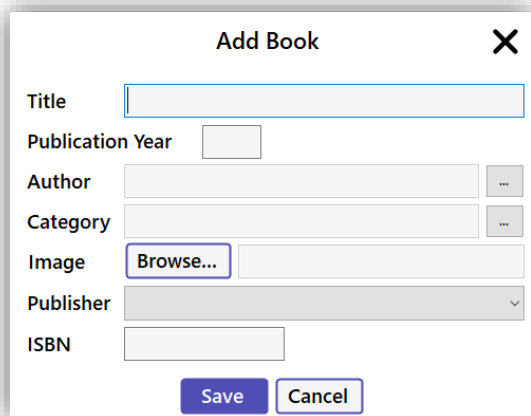
Nagyon fontos, hogy amelyik kölcsönzés vagy foglalás egy könyvhöz kapcsolódik, azok szintén törlésre kerülnek!

Továbbá van egy „**keresőmezőnk**”, ahol a könyv tulajdonságai alapján tudunk keresni a táblázatban, ezzel könnyebbé téve a könyvek kezelését. Az említett könyvek pedig az imént említett gombok alatti táblázatban kerülnek kilistázásra.



Title	Author	Publication Year	Category	Publisher	ISBN
1984	George Orwell	1949	Fiction, Science Fiction	Penguin Random House	9780451524935
Animal Farm	George Orwell	1945	Classics, Dystopian	Penguin Random House	9780141187761
Brave New World	Aldous Huxley	1932	Dystopian	Cengage	9780060850524
Crime and Punishment	Fyodor Dostoevsky, Haruki Murakami	1866	Fiction	Macmillan Publishers	9780486415871
Frankenstein	Mary Shelley	1818	Horror	McGraw-Hill	9780141439563
Great Expectations	Charles Dickens	1861	Fiction	Cambridge University Press	9780486415864
Hamlet	Isabel Allende, William Shakespeare	1603	Fiction	Routledge	9780743477123
Harry Potter and the Sorcerer's Stone	J.K. Rowling	1997	Fantasy, Young Adult	HarperCollins	9780439708180
Les Misérables	Victor Hugo	1862	Historical Fiction	Routledge	9780140449143
Moby Dick	Herman Melville	1851	Classic Literature	Wiley	9780143105428
Murder on the Orient Express	Agatha Christie, George Orwell	1934	Mystery	McGraw-Hill	9780062693661
Norwegian Wood	Haruki Murakami	1987	Fiction	Oxford University Press	9780375704024
One Hundred Years of Solitude	Gabriel Garcia Marquez	1967	Fantasy, Fiction	Hachette Livre	9780060883287
Pride and Prejudice	Jane Austen	1813	Romance	Scholastic	9780141040349
The Adventures of Tom Sawyer	Isabel Allende, Mark Twain	1876	Fiction, Self-help	Wiley	9780486400778
The Brothers Karamazov	Fyodor Dostoevsky	1880	Coming-of-Age	Hachette Livre	9780140449266

ábra 118: Könyvek



Add Book [X]

Title:

Publication Year:

Author: ...

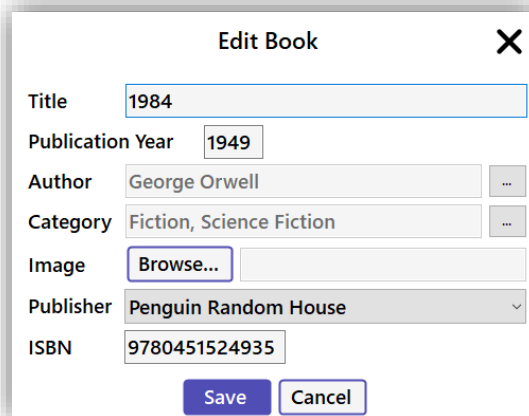
Category: ...

Image:

Publisher: v

ISBN:

ábra 119: Könyv hozzáadása



Edit Book [X]

Title:

Publication Year:

Author: ...

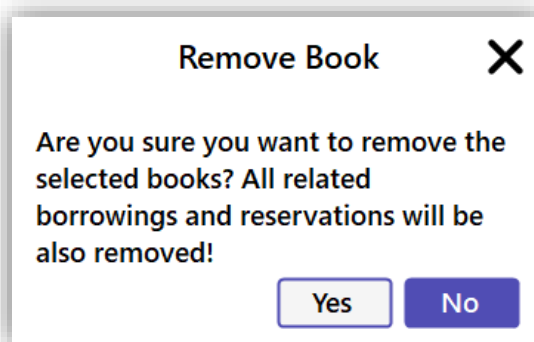
Category: ...

Image:

Publisher: v

ISBN:

ábra 120: Könyv szerkesztése



Remove Book [X]

Are you sure you want to remove the selected books? All related borrowings and reservations will be also removed!

ábra 121: Könyv eltávolítása

Kölcsönzések (Borrowings)

A „**kölcsönzések**” fül szintén egy frissítés gombbal kezdődik, ami lefrissíti a kölcsönzéseket az adatbázisból, kilistázva azokat a gombok alatt található táblázatba.

Ezt követi egy „**kölcsönbe adás**” gomb, mely megnyit egy ablakot (11. ábra), amiben ki tudjuk választani, hogy melyik felhasználónak, milyen könyvet és mennyi időre szeretnénk kölcsönbe adni.

A **könyvek kiválasztására** külön felület van (13. ábra), ahol a bal oldalon jelennek meg a szabad könyvek, a jobb oldalra pedig amiket kiválasztunk kiadásra. A következő gomb egy meghosszabbítás, ahol megadható a meghosszabbítás időtartama (12. ábra).

A visszahozott könyveket a jobbra lévő gombbal tudjuk **megjelölni visszahozottként** egy megerősítést követően (14. ábra).

Ezen a felületen is helyet kapott egy „**keresőmező**”, ami a kölcsönzések között keres. Ezenfelül egy jelölőnégyzet is helyet kapott, melyre bepipálásával szűrhetünk azokra a kölcsönzésekre, amelyek jelenleg is aktívak, azaz még nem hozták vissza az adott könyvet.



Lend
Extend
Return

Borrowings

Only show current borrowings ☐

Username	Title	ISBN	Borrow Date	Due Date	Return Date
alicej	Moby Dick	9780143105428	30/10/2024	13/11/2024	
librarian1	The Hobbit	9780547928227	28/10/2024	11/11/2024	
admin	The Catcher in the Rye	9780141187396	26/10/2024	09/11/2024	
janesmith	Wuthering Heights	9780141439518	24/10/2024	07/11/2024	
johndoe	Animal Farm	9780141187761	22/10/2024	05/11/2024	
alicej	Moby Dick	9780143105428	15/10/2024	29/10/2024	28/10/2024
librarian1	Brave New World	9780060850524	14/10/2024	28/10/2024	
librarian1	The Hobbit	9780547928227	13/10/2024	27/10/2024	26/10/2024
admin	To Kill a Mockingbird	9780061120084	12/10/2024	26/10/2024	
admin	The Catcher in the Rye	9780141187396	11/10/2024	25/10/2024	24/10/2024
janesmith	The Brothers Karamazov	9780140449266	10/10/2024	24/10/2024	
janesmith	Wuthering Heights	9780141439518	09/10/2024	23/10/2024	22/10/2024
johndoe	The Great Gatsby	9780743273565	08/10/2024	22/10/2024	
johndoe	Animal Farm	9780141187761	07/10/2024	21/10/2024	20/10/2024
lauraharris	The Trial	9780805209990	06/10/2024	20/10/2024	
jackyoung	Great Expectations	9780486415864	05/10/2024	19/10/2024	

ábra 122: Kölcsönzések

Lend book

User

Books

Due Time

1 Month

Save

Cancel

Extend Borrowing

Extend by

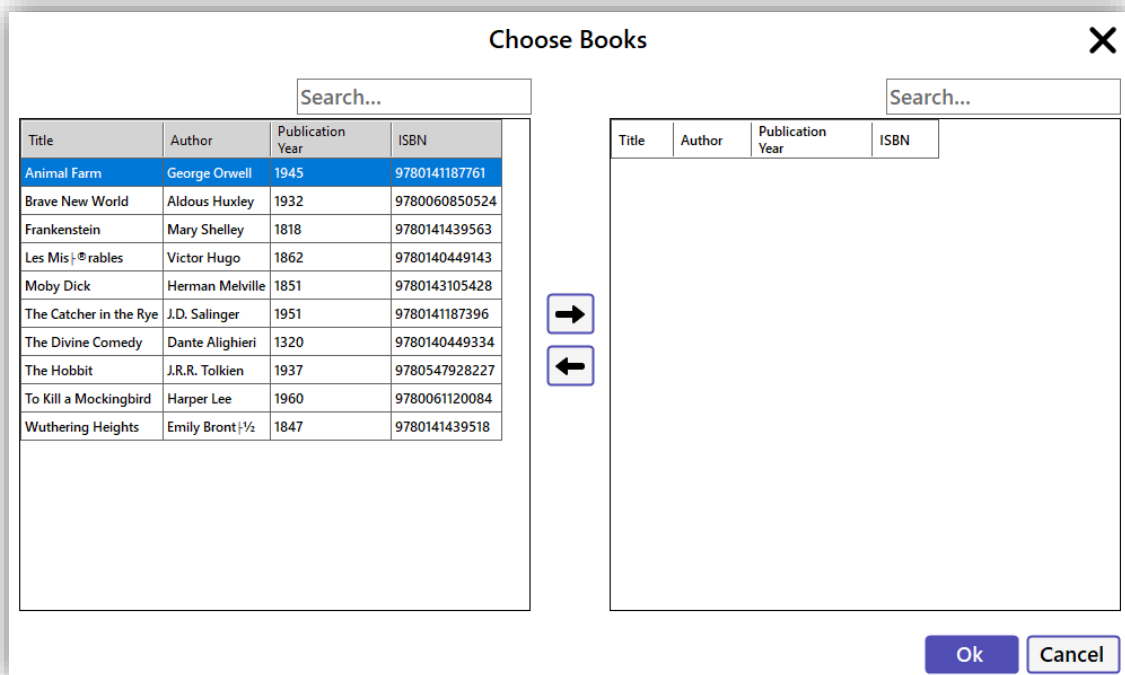
1 Month

Save

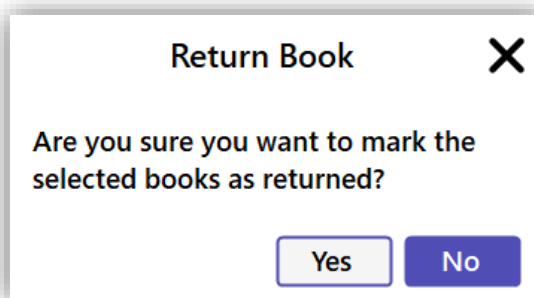
Cancel

ábra 123: Kölcsönbe adás

ábra 124: Kölcsönzés meghosszabbítása



ábra 125: Könyvek kiválasztása



ábra 126: Könyv visszahozása

Foglalások (Reservations)

A „**foglalásokra**” kattintva megjelenik a szokásos táblázat, ami feltöltésre kerül az adatbázisban tárolt foglalásokkal (15. ábra).

Ezt a táblázatot tudjuk lefrissíteni a bal felső sarokban található „**frissítés**” gombbal.

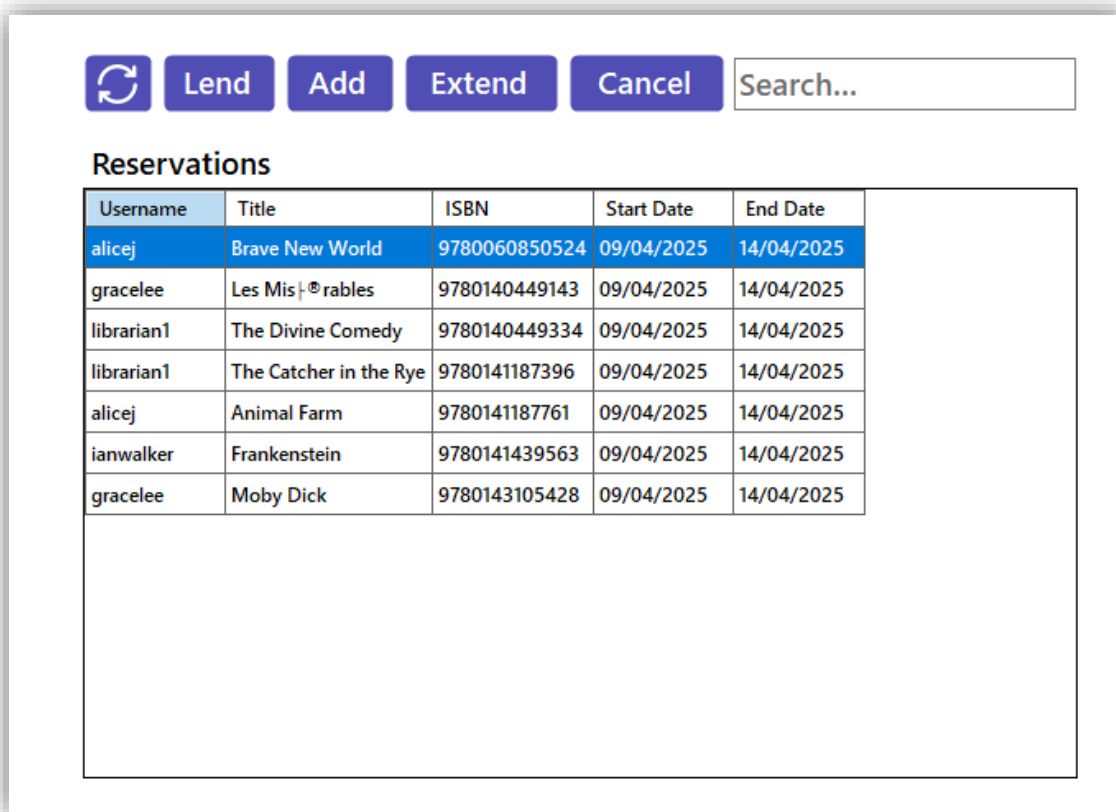
Ezt követi egy „**kölcsönbe adás**” gomb, melynek megnyomásakor egy megerősítést követően (16. ábra) kölcsönbe tudjuk adni a lefoglalásban szereplő könyvet.

A „**hozzáadás**” gombbal tudunk hozzáadni egy új foglalást a megnyíló ablakban (17. ábra), ami lefoglalja az adott könyvet és az nem adható ki a foglalás idejére. Ez a foglalás automatikusan törlődik **5 nap** után.

A „**meghosszabbítás**” gombbal ez elkerülhető, amit megerősítés után (18. ábra) **5 napra** tudunk meghosszabbítani.

Végül van egy „**lemondás**” gombunk, ami az adott foglalást semmissé teszi, eltávolítja az adatbázisból a megerősítést követően (19. ábra), így más ki tudja kölcsönözni, vagy lefoglalni azt.

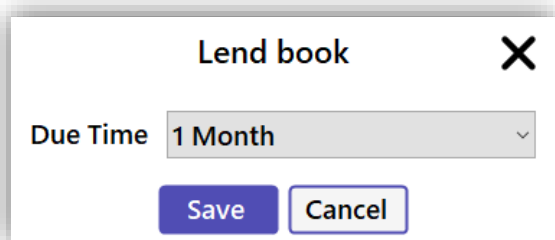
Végül ezen az oldalon is megtalálható egy „**keresőmező**”, ami keres a kilistázott foglalásokban.



The screenshot shows a web interface for managing reservations. At the top, there are five buttons: a refresh icon, "Lend", "Add", "Extend", and "Cancel", followed by a search input field labeled "Search...". Below these is a section titled "Reservations" containing a table with the following data:

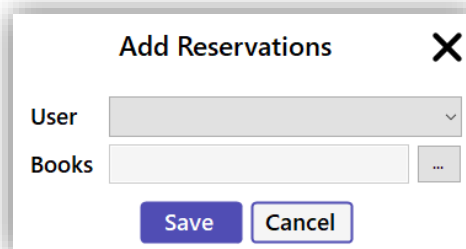
Username	Title	ISBN	Start Date	End Date
alicej	Brave New World	9780060850524	09/04/2025	14/04/2025
gracelee	Les Misérables	9780140449143	09/04/2025	14/04/2025
librarian1	The Divine Comedy	9780140449334	09/04/2025	14/04/2025
librarian1	The Catcher in the Rye	9780141187396	09/04/2025	14/04/2025
alicej	Animal Farm	9780141187761	09/04/2025	14/04/2025
ianwalker	Frankenstein	9780141439563	09/04/2025	14/04/2025
gracelee	Moby Dick	9780143105428	09/04/2025	14/04/2025

ábra 127: Foglalások



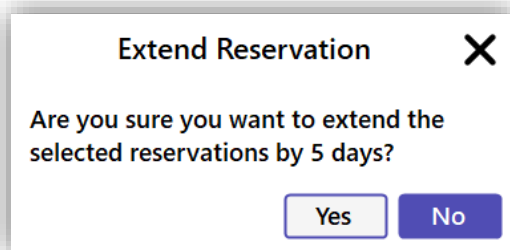
The "Lend book" dialog box has a title bar with a close button (X). It contains a label "Due Time" followed by a dropdown menu currently showing "1 Month". At the bottom are two buttons: "Save" and "Cancel".

ábra 128: Kölcsönbe adás

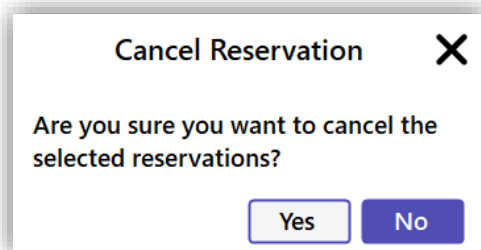


The "Add Reservations" dialog box has a title bar with a close button (X). It contains two input fields: "User" (a dropdown menu) and "Books" (a text input field with a search icon). At the bottom are two buttons: "Save" and "Cancel".

ábra 129: Foglalás hozzáadása



ábra 130: Foglalás meghosszabbítása



ábra 131: Foglalás visszamondása

Kategóriák (Categories)

A „**kategóriákra**” kattintva megjelenik a szokásos táblázat, ami ezúttal az adatbázisban tárolt kategóriákkal töltődik meg (20. ábra).

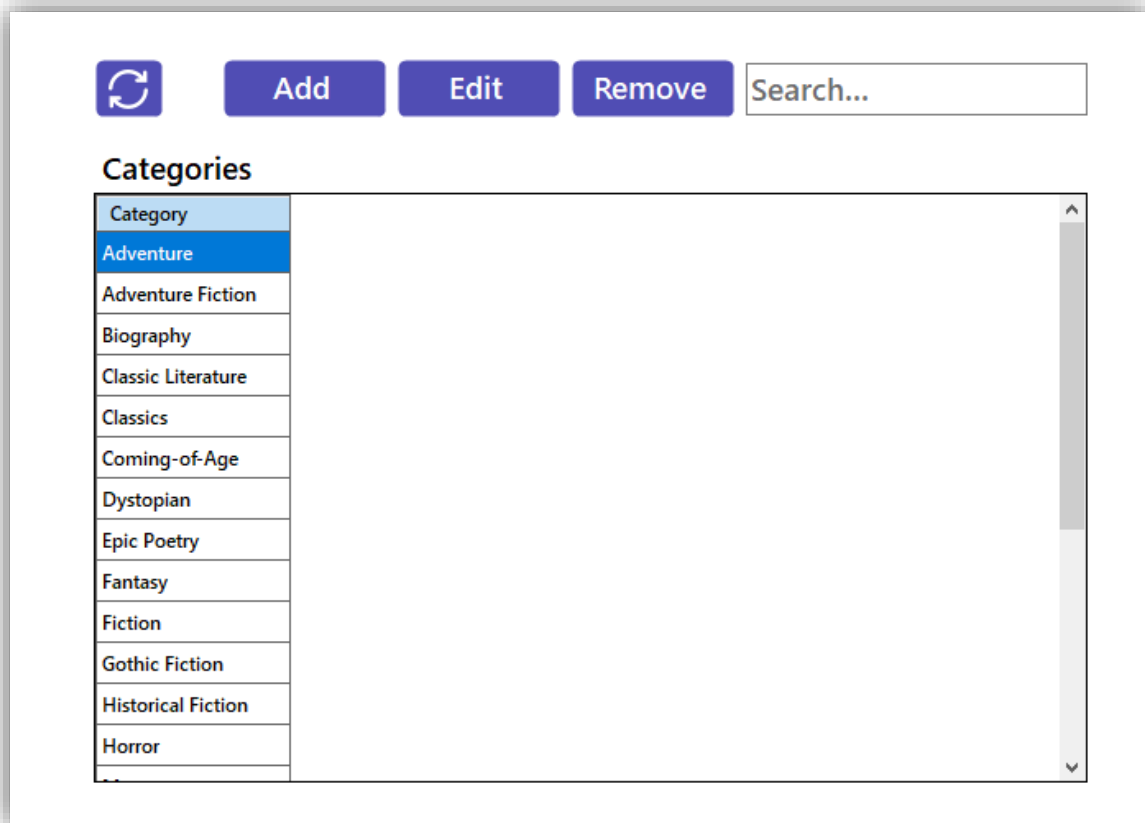
A „**hozzáadás**” gombbal egész egyszerűen hozzá tudunk adni egy kategóriát szimplán a kategória nevének megadásával (21. ábra).

A kategória **módosítása** hasonlóan működik, csak itt alaphól betöltésre kerül a kategória régi neve a megnyíló ablakba (22. ábra).

És végül az „**eltávolítás**” gombra kattintva, egy megerősítést követően (23. ábra) eltávolíthatjuk az adott kategóriát az adatbázisból.

Nagyon fontos, hogy a törlést követően az ezzel a kategóriával rendelkező könyvek is törölődnek!

Ezen az oldalon szintén megtalálható a „**keresőmező**”, ahol a beírt szöveg alapján lehet keresni a kategóriák között.



ábra 132: Kategóriák

Add Category
✕

Category

Save Cancel

ábra 133: Kategória hozzáadása

Edit Category
✕

Category

Save Cancel

ábra 134: Kategória szerkesztése

Remove Category
✕

Are you sure you want to remove the selected categories? All related books, borrowings and reservations will be also removed!

Yes No

ábra 135: Kategória eltávolítása

Szerzők (Authors)

A „**szerzőkre**” kattintva kilistázásra kerül a táblázatba az összes adatbázisban tárolt szerző (24. ábra).

A bal felső sarokban található „**frissítés**” gomb ezeket az adatokat frissíti az adatbázisból.

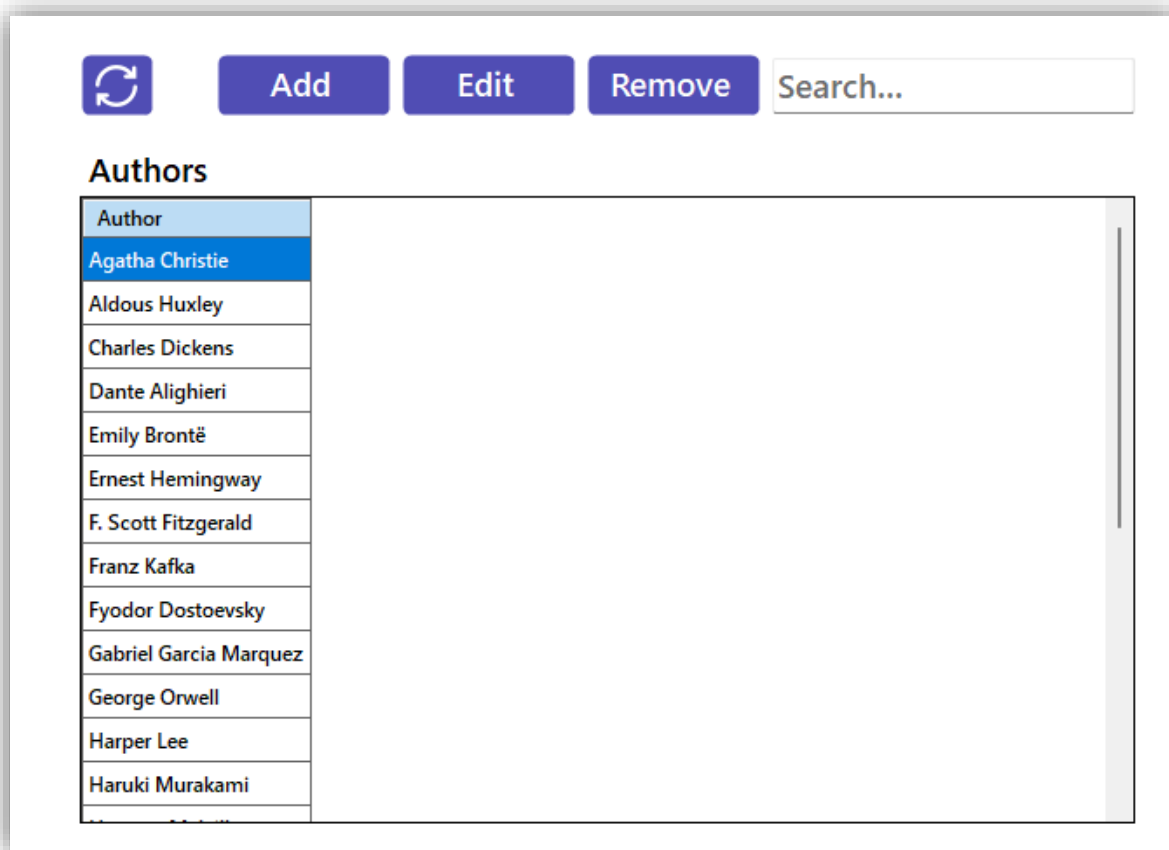
A következő, „**hozzáadás**” gomb megnyit nekünk egy ablakot (25. ábra), ahol a szerző nevének megadása után bekerül az adatbázisba.

Ezt a megszokottak szerint egy „**módosítás**” gomb követi, ahol a kiválasztott szerző betöltésre kerül egy új ablakban (26. ábra), és ezt tudjuk szerkeszteni, hogy utána bekerüljön a frissített változat az adatbázisba.

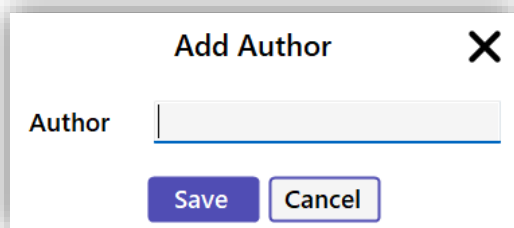
Az utolsó gombunk a „**törlés**”, ami egy megerősítést követően kitörli a kiválasztott szerzőket (27. ábra).

Nagyon fontos, hogy azok a könyvek, amelyekhez ez a szerző volt hozzárendelve törlésre fognak kerülni!

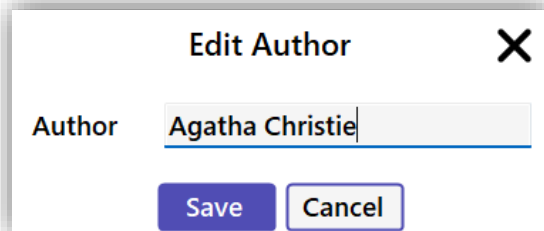
Végül van egy „**keresőmezőnk**”, ami a szerzők neve alapján keres a lenti táblázatban.



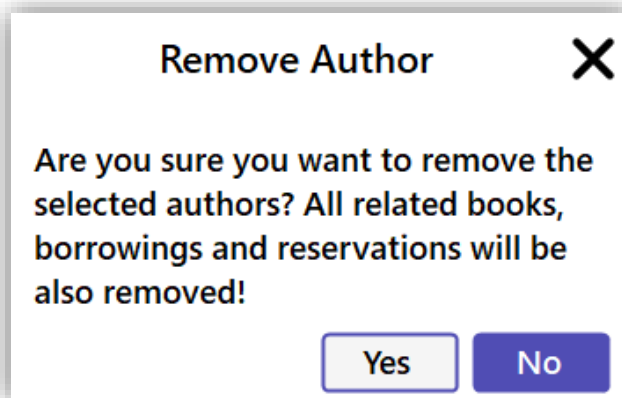
ábra 136: Szerzők

A dialog box titled "Add Author" with a close button (X) in the top right corner. It contains a label "Author" followed by a text input field. At the bottom, there are two buttons: "Save" and "Cancel".

ábra 137: Szerző hozzáadása

A dialog box titled "Edit Author" with a close button (X) in the top right corner. It contains a label "Author" followed by a text input field containing the text "Agatha Christie". At the bottom, there are two buttons: "Save" and "Cancel".

ábra 138: Szerző szerkesztése

A dialog box titled "Remove Author" with a close button (X) in the top right corner. The main text reads: "Are you sure you want to remove the selected authors? All related books, borrowings and reservations will be also removed!". At the bottom, there are two buttons: "Yes" and "No".

ábra 139: Szerző eltávolítása

Kiadók (Publishers)

A „**kiadók**” oldalra kattintva megjelenik a szokásos táblázat (28. ábra) ezúttal a felhasználókkal feltöltve.

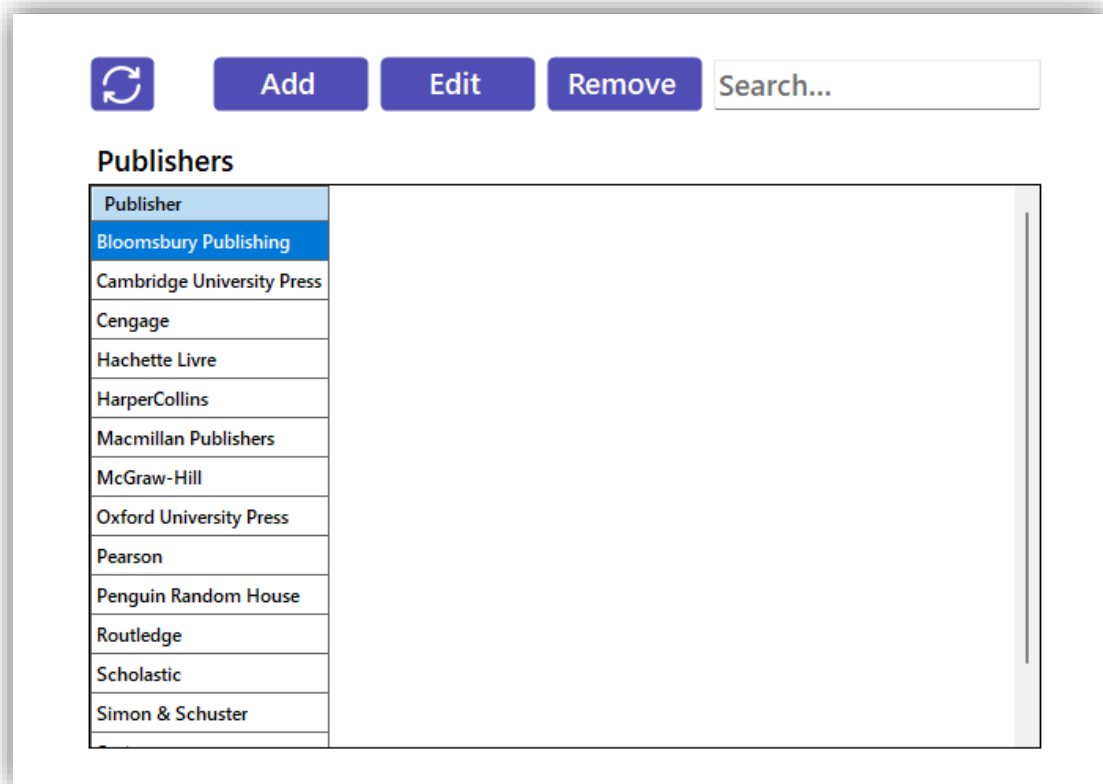
Ezt követi egy „**hozzáadás**” gomb, amely megnyitja a hozzáadáshoz szükséges ablakot (29. ábra). Itt fel tudjuk venni az új kiadót.

A „**módosítás**” gomb megnyitja a módosítás ablakot (30. ábra), ahol betöltésre kerül a kiválasztott kiadó és az új kiadó nevét frissíti az adatbázisban.

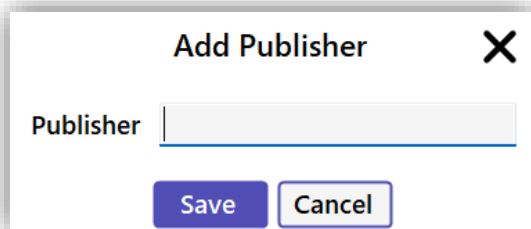
Az „**eltávolítás**” gomb pedig a kiválasztott kiadó(ka)t eltávolítja az adatbázisból egy megerősítést követően (31. ábra)

Nagyon fontos, hogy azok a könyvek, amik kapcsolva vannak az adott szerző(k)höz szintén törölve lesznek!

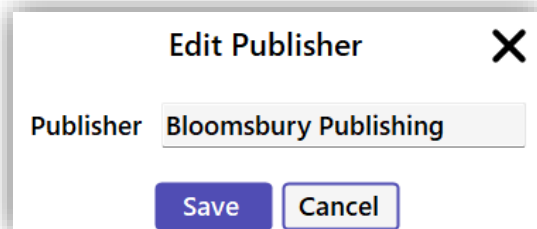
És végül ezen az oldalon is helyet kapott egy „**keresőmező**”, ahol lehet keresni az adatbázisban tárol kiadók között.



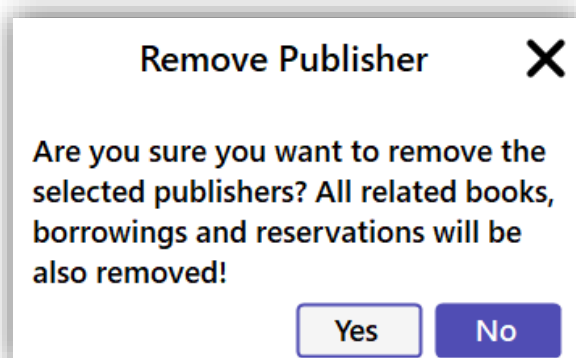
ábra 140: Kiadók



ábra 141: Kiadó hozzáadása



ábra 142: Kiadó szerkesztése



ábra 143: Kiadó eltávolítása

Felhasználók (Users)

A „**felhasználók**” fül alatt szintén kilistázásra kerülnek a felhasználók és az adataik egy táblázatban (32. ábra)

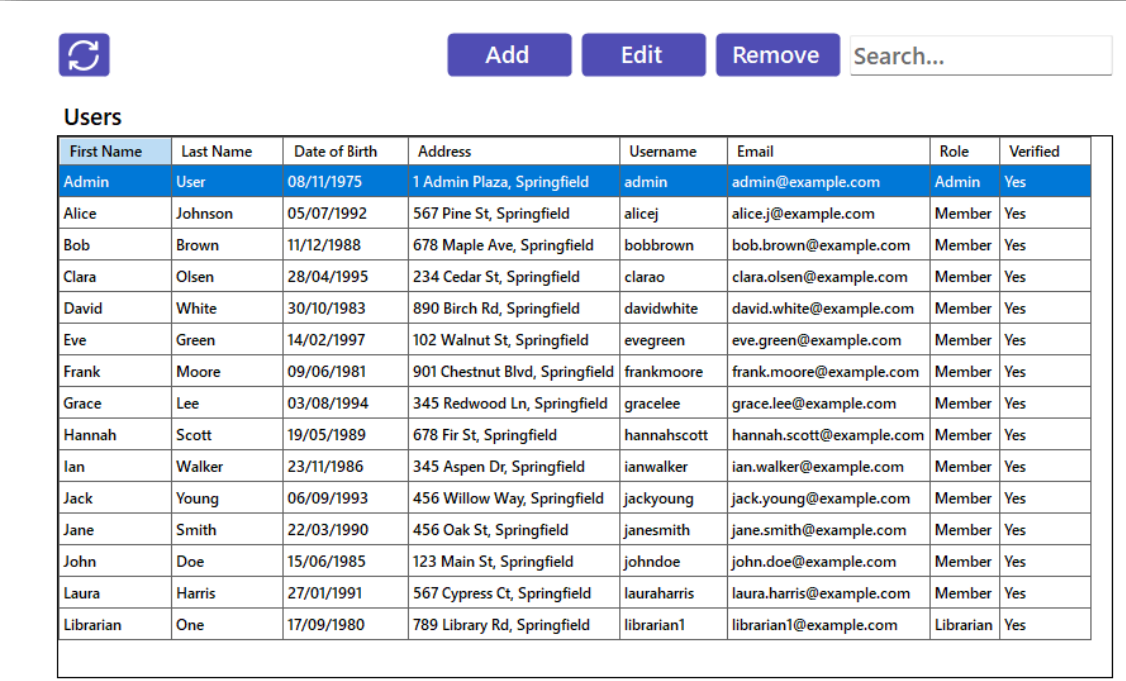
Az első gomb a felhasználó **hozzáadása**, mely megnyitja az adatlapot (33. ábra), aminek kitöltésével fel tudunk venni egy új felhasználót. Itt a rang opció csak akkor jelenik meg, ha adminisztrátorként vagyunk bejelentkezve. A felhasználó felvételét követően megjelenik a felhasználó véletlenszerűen generált új jelszava (36. ábra), amit a webes felületen tetszés szerint módosíthat.

A „**szerkesztés**” gomb megnyomásával megnyílik a fent említett adatlap (34. ábra), viszont ezúttal betölti a kijelölt felhasználó adatait. A szerkesztés csak akkor elérhető, ha adminisztrátorként vagyunk bejelentkezve! Ha a felhasználó elfelejtette a jelszavát, lehetősége van új leszót generáltatni a könyvtárossal, ami egy véletlenszerűen generált jelszó, ami szintén a felugró ablakban jelenik meg (36. ábra).

Az „**eltávolítás**” gomb pedig eltávolítja a kiválasztott felhasználó(ka)t a megerősítést követően (35. ábra). Ez az opció szintén csak akkor jelenik meg, ha adminisztrátorként vagyunk bejelentkezve!

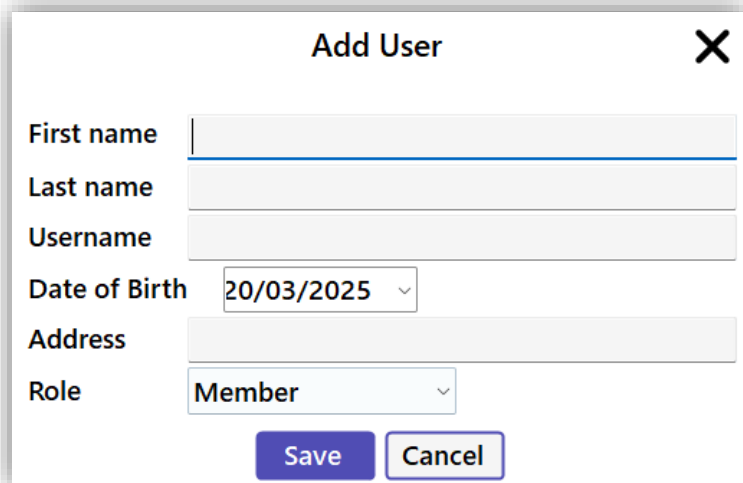
Nagyon fontos, hogy azok a kölcsönzések és foglalások, amelyek a kijelölt felhasználó(k)hoz tartozik/tartoznak szintén törlésre fognak kerülni!

Végül pedig a „**keresőmező**”, ami ennél a fülnél is a táblázatban keres.



First Name	Last Name	Date of Birth	Address	Username	Email	Role	Verified
Admin	User	08/11/1975	1 Admin Plaza, Springfield	admin	admin@example.com	Admin	Yes
Alice	Johnson	05/07/1992	567 Pine St, Springfield	alicej	alice.j@example.com	Member	Yes
Bob	Brown	11/12/1988	678 Maple Ave, Springfield	bobbrown	bob.brown@example.com	Member	Yes
Clara	Olsen	28/04/1995	234 Cedar St, Springfield	clarao	clara.olsen@example.com	Member	Yes
David	White	30/10/1983	890 Birch Rd, Springfield	davidwhite	david.white@example.com	Member	Yes
Eve	Green	14/02/1997	102 Walnut St, Springfield	evegreen	eve.green@example.com	Member	Yes
Frank	Moore	09/06/1981	901 Chestnut Blvd, Springfield	frankmoore	frank.moore@example.com	Member	Yes
Grace	Lee	03/08/1994	345 Redwood Ln, Springfield	gracelee	grace.lee@example.com	Member	Yes
Hannah	Scott	19/05/1989	678 Fir St, Springfield	hannahscott	hannah.scott@example.com	Member	Yes
Ian	Walker	23/11/1986	345 Aspen Dr, Springfield	ianwalker	ian.walker@example.com	Member	Yes
Jack	Young	06/09/1993	456 Willow Way, Springfield	jackyoung	jack.young@example.com	Member	Yes
Jane	Smith	22/03/1990	456 Oak St, Springfield	janesmith	jane.smith@example.com	Member	Yes
John	Doe	15/06/1985	123 Main St, Springfield	johndoe	john.doe@example.com	Member	Yes
Laura	Harris	27/01/1991	567 Cypress Ct, Springfield	lauraharris	laura.harris@example.com	Member	Yes
Librarian	One	17/09/1980	789 Library Rd, Springfield	librarian1	librarian1@example.com	Librarian	Yes

ábra 144: Felhasználók



Add User [X]

First name

Last name

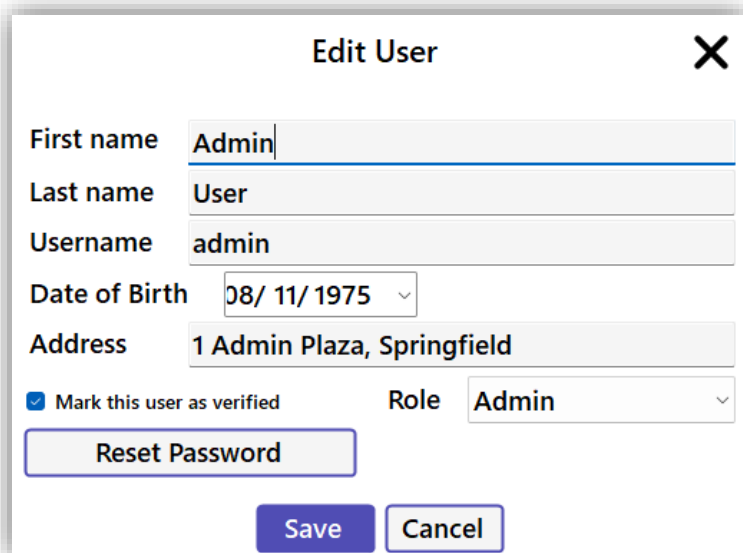
Username

Date of Birth

Address

Role

ábra 145: Felhasználó hozzáadása



Edit User [X]

First name

Last name

Username

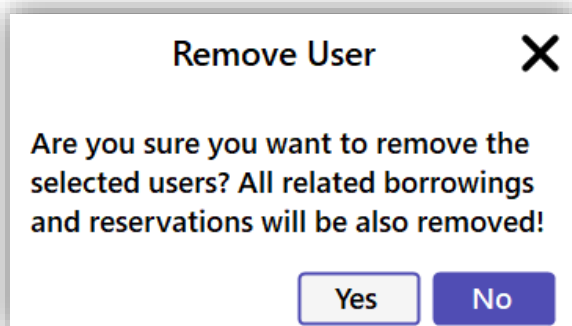
Date of Birth

Address

☒ Mark this user as verified

Role

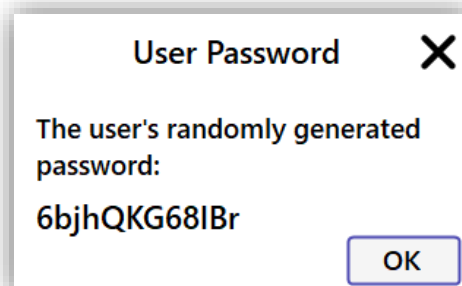
ábra 146: Felhasználó szerkesztése



Remove User [X]

Are you sure you want to remove the selected users? All related borrowings and reservations will be also removed!

ábra 147: Felhasználó eltávolítása



User Password [X]

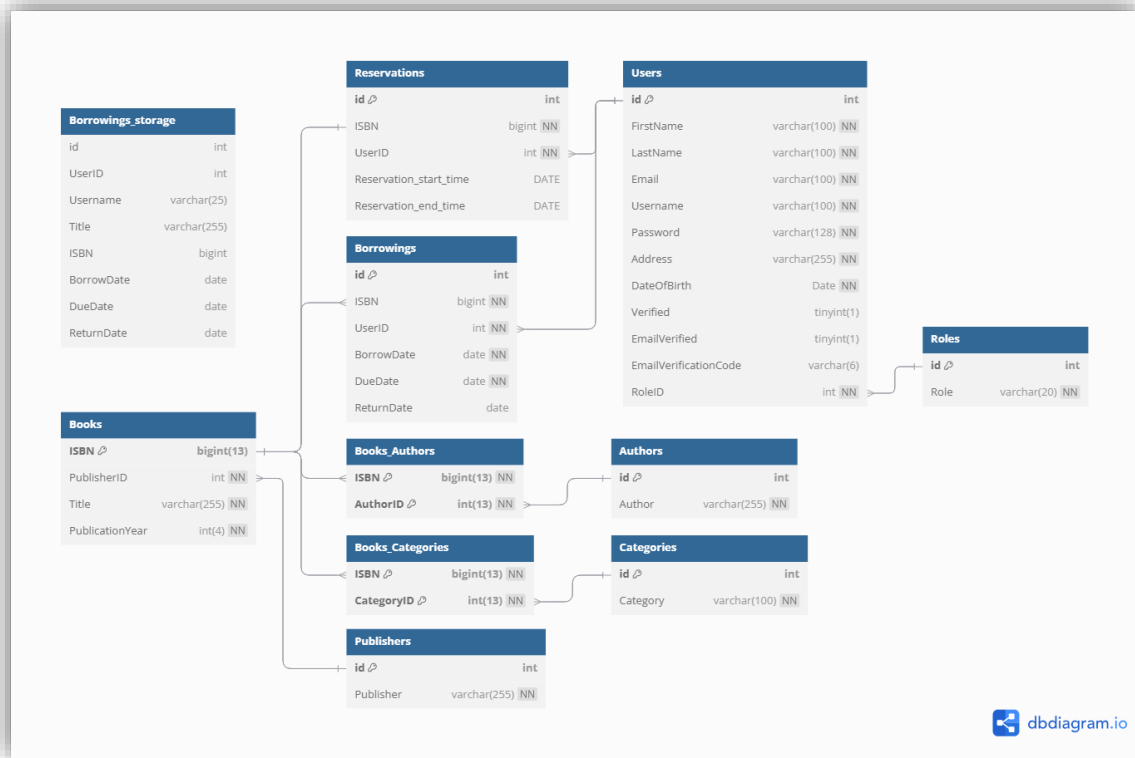
The user's randomly generated password:

6bjhQKG68lBr

ábra 148: Generált jelszó

Fejlesztői dokumentáció

Adatbázis



ábra 149: Adatbázis diagram

Az adatbázis importálható a .../Database/Database.sql állományból. Az adatbázisba minta adatokat és felhasználókat szintén tartalmazza az sql állomány.

Books tábla

- ISBN: Elsődleges kulccsal rendelkezik, 13 számjegyű bigint. Ezzel azonosítjuk egyedileg a könyveket.
- PublisherID: Idegen kulcs, Publishers táblához van kötve, amiben tároljuk a kiadókat.
- Title: A könyv címe, 255 karakter hosszú string.
- PublicationYear: A könyv kiadásának dátuma, 4 számjegyű int.

Publishers tábla

- Id: Elsődleges kulcs
- Publisher: A kiadó neve, 255 hosszú string.

Categories tábla

- Id: Elsődleges kulcs
- Category: A kategória neve
- A táblában, ha törlésre kerül egy adat, az törlődik a Books_ Categories táblából is.

Books_Categories tábla

- Ez egy kapcsolótábla, ami összeköti a könyv ISBN-jét a kategóriák tábla azonosítójával.

Authors tábla

- Id: Elsődleges kulcs
- Author: A szerző neve
- A táblában, ha törlésre kerül egy adat, az törlődik a Books_Authors táblából is.

Books_Authors tábla

- Kapcsolótábla, összeköti a könyv ISBN számát az Authors tábla azonsítójával.

Roles tábla

- Id: Elsődleges kulcs
- Role: Szerepeket tárol, előre meghatározva: Admin, Librarian, Member

Users tábla

- Id: Elsődleges kulcs
- FirstName: 100 karakter hosszú string.
- LastName: 100 karakter hosszú string.
- Email: 100 karakter hosszú string.
- Username: 100 karakter hosszú string.
- Password: 128 karakter hosszú string. Azért 128, mert minden esetben ilyen hosszú a titkosított jelszó.
- Address: 255 karakter hosszú string.
- DateOfBirth: Date típusú dátum.
- Verified: tinyint, lehet 0 vagy 1, ami meghatározza, hogy a felhasználó adatai meg lettek-e erősítve.

- EmailVerified: tinyint, lehet 0 vagy 1, ami meghatározza, hogy az adott felhasználónak meg van-e erősítve az email címe.
- EmailVerificationCode: 6 karakter hosszú string, itt tárolja ideiglenesen a felhasználó email megerősítési kódját.
- RoleID: A Roles tábla id mezőjére hivatkozó idegen kulcs.

Borrowings tábla

- Id: Elsődleges kulcs
- ISBN: Idegen kulcs, a könyv ISBN számára hivatkozik.
- UserID: Idegen kulcs, a felhasználó azonosítójára hivatkozik.
- BorrowDate: Date típusú.
- DueDate: Date típusú.
- ReturnDate: Date típusú, ha üres, akkor még nincs visszahozva az adott könyv.

Reservations tábla

- Id: Elsődleges kulcs.
- ISBN: Idegen kulcs, a könyv ISBN számára hivatkozik.
- UserID: Idegen kulcs, a felhasználó azonosítójára hivatkozik.
- Reservation_start_time: Date típusú, a foglalás dátuma.
- Reservations_end_time: Date típusú, a foglalás végének dátuma.

Borrowings_storage tábla

- A Borrowings table biztonsági másolata, nem kapcsolódik semmilyen táblához. Minden adat tárolva van benne, ami ahhoz szükséges, ha egy könyv törlésre kerül, az a kölcsönzések között megmaradjon, ezáltal visszakereshető legyen.

after_borrowings_insert trigger

- Amikor a Borrowings táblába valami feltöltésre kerül, automatikusan a Borrowings_storage-be is feltöltésre fog kerülni

after_borrowings_update trigger

- Amikor a Borrowings táblában valami frissítésre kerül, automatikusan frissül a Borrowings_storage táblában is.

delete_authors_and_books trigger

- Mivel, ha egy szerzőt törölünk, az automatikusan törli a kapcsolótáblából is az adatot, viszont így a könyv szerző nélkül marad, hibát eredményezve. Ezért, ha az Authors táblából valami törlésre kerül, a hozzá tartozó könyv is törlésre fog kerülni a Books táblából.

delete_categories_and_books trigger

- Mivel, ha egy kategóriát törölünk, az automatikusan törli a kapcsolótáblából is az adatot, viszont így a könyv kategória nélkül marad, hibát eredményezve. Ezért, ha a Categories táblából valami törlésre kerül, a hozzá tartozó könyv is törlésre fog kerülni a Books táblából.

delete_expired_reservation event

- Ez az event felel azért, hogyha egy foglalás több ideje van bent, mint 5 nap, akkor automatikusan törlésre kerül.

Backend

Router.php

Ez a fájl a ``Router`` osztályt definiálja, amely a HTTP kérések kezeléséért és a megfelelő vezérlőkhöz (controller) való irányításáért felelős. Támogatja a többféle HTTP metódust (GET, POST, PUT, DELETE), valamint biztosítja a hitelesítést, fájlkezelést és az URL-ekből történő változók kinyerését.

Funkciók:

- ``post()``: POST kérések kezelése egy adott végpontra.
- ``get()``: GET kérések kezelése egy adott végpontra, opcionális URL-változó kinyeréssel.
- ``put()``: PUT kérések kezelése egy adott végpontra.
- ``delete()``: DELETE kérések kezelése egy adott végpontra.
- Hitelesítés: Token alapú hitelesítés támogatása védett útvonalakhoz.
- URL feldolgozás: Változók és adatok kinyerése az URL-ből dinamikus végpontokhoz.
- Segédfüggvények integrálása: Segédfüggvények használata bemenetellenőrzéshez és a kérés törzsének feldolgozásához.

Használat:

- Az útvonalakat a statikus metódusok (``post``, ``get``, ``put``, ``delete``) meghívásával lehet definiálni, megadva a végpontot, a vezérlőt, a függvényt és opcionálisan további paramétereket (pl. hitelesítés).
- A vezérlőknek implementálniuk kell a megadott függvényeket, hogy kezelni tudják az irányított kéréseket.

Függőségek:

- ``Helper\Helper``: Segédfüggvényeket biztosít a bemenetek ellenőrzéséhez és a kérés törzsének feldolgozásához.
- ``App\Authorize\Token``: A tokenek ellenőrzéséért felelős a hitelesítés során.

Helper.php

Ez a fájl a `Helper` osztályt definiálja, amely segédfüggvényeket biztosít a bemenetek érvényesítéséhez és a kérés törzsének feldolgozásához. Célja a HTTP kérésekből érkező adatok épségének és biztonságának biztosítása.

Funkciók:

- `validateTheInput()`: Egyetlen bemeneti érték érvényesítése és megtisztítása.
- `validateTheInputArray()`: Több bemeneti értékből álló tömb érvényesítése és megtisztítása.
- `getPostBody()`: A nyers POST kérés törzsének beolvasása és dekódolása, opcionálisan JSON-formátum ellenőrzésével.

Használat:

- A `validateTheInput()` függvényt egyéni bemeneti értékek megtisztítására használd.
- A `validateTheInputArray()` segítségével tömbök, például lekérdezési paraméterek vagy URL-változók tisztíthatók.
- A `getPostBody()` használható POST kérések JSON törzsének értelmezésére és érvényesítésére.

Függőségek:

- `ApiResponse\Response`: A HTTP hibaválaszok kezelésére szolgál, ha az érvényesítés sikertelen.

SendEmail.php

Ez a fájl a PHPMailer alapfájlja. A forráskód elérhető itt:

<https://github.com/PHPMailer/PHPMailer> Ez a fájl kezeli az e-mailek küldését. A projekt e-mail címe: librarymanagementsystem.emailer@gmail.com, így minden, a rendszer által küldött e-mail erről a címről fog érkezni.

Funkciók:

- A `sendEmail` függvény 4 paramétert vár: címzett, címzett neve, tárgy, szöveg
- `sendEmail()`: E-mail küldése a szoftver e-mail címéről

Használat:

- A ``sendEmail()``: függvényt egy e-mail elküldésére használd, ehhez definiáld egy HTML szöveget előtte

Függőségek:

- PHPMailer\PHPMailer az e-mail küldéséhez szükséges funkciókat és osztályokat tartalmazza
- ``ApiResponse\Response``: A HTTP hibaválaszok kezelésére szolgál, ha az érvényesítés sikertelen.

BooksTable.php

Ez a fájl a ``BooksTable`` osztályt definiálja, amely a ``Table`` osztályból származik, és kifejezetten a könyvekkel kapcsolatos adatok kezelésére és lekérdezésére biztosít metódusokat.

Funkciók:

- ``selectByParams()``: Könyvek lekérdezése megadott mezők, operátorok, értékek és típusok alapján.
- ``countAllBooks()``: Az adatbázisban található összes könyv megszámlálása.
- ``selectByISBN()``: Egy könyv adatainak lekérdezése ISBN alapján.
- ``availableBooks()``: Egy adott ISBN-hez tartozó könyv elérhetőségének ellenőrzése.
- ``selectAvailableBooks()``: Azoknak a könyveknek a listázása, amelyek jelenleg elérhetők (nincsenek kikölcsönözve vagy lefoglalva).

Használat:

- Ezekkel a metódusokkal lehet kapcsolatba lépni a ``books`` táblával és a hozzá kapcsolódó táblákkal (pl. ``publishers``, ``authors``, ``categories``).
- A ``selectByParams()`` és ``selectAvailableBooks()`` metódusok támogatják az összetett lekérdezéseket join-olással és csoportosítással.

Függőségek:

- A ``Table`` osztályból származik, amely az alap lekérdezésépítési funkciókat biztosítja.
- Használja a kapcsolódó adatbázistáblákat, mint például: ``books``, ``publishers``, ``authors``, ``categories``, ``borrowings`` és ``reservations``.

BorrowingsTable.php

Ez a fájl a ``BorrowingsTable`` osztályt definiálja, amely a ``Table`` osztályból származik, és kifejezetten a kölcsönzésekkel kapcsolatos adatok kezelésére és lekérdezésére biztosít metódusokat.

Funkciók:

- ``allBorrowings()``: Az összes kölcsönzés megszámlálása az adatbázisban.
- ``bookInBorrowings()``: Annak ellenőrzése, hogy egy adott könyvet (ISBN alapján) jelenleg kölcsönözték-e, és még nem került vissza.
- ``selectMyBooks()``: Egy adott felhasználó által kölcsönzött könyvek listázása, a kölcsönzés részleteivel együtt.
- ``selectTopBorrowedBook()``: A legtöbbször kölcsönzött könyvek lekérdezése, megadott darabszámig korlátozva.

Használat:

- Ezekkel a metódusokkal lehet kapcsolatba lépni a ``borrowings`` táblával és a kapcsolódó táblákkal (pl. ``books``, ``authors``, ``categories``).
- A ``selectMyBooks()`` és ``selectTopBorrowedBook()`` metódusok támogatják az összetett lekérdezéseket join-okkal, csoportosítással és rendezéssel.

Függőségek:

- A ``Table`` osztályból származik, amely az alap lekérdezésépítési funkciókat biztosítja.
- Használja a kapcsolódó adatbázistáblákat, mint például: ``borrowings``, ``books``, ``authors``, ``categories``, és ``publishers``.

ReservationTable.php

Ez a fájl a ``ReservationTable`` osztályt definiálja, amely a ``Table`` osztályból származik, és kifejezetten a foglalásokkal kapcsolatos adatok kezelésére és lekérdezésére biztosít metódusokat.

Funkciók:

- ``reserve()``: Új foglalás létrehozása egy adott könyvre (ISBN alapján) és felhasználóra.
- ``deleteReservation()``: Egy adott könyvre (ISBN alapján) vonatkozó foglalás törlése.
- ``bookInReservation()``: Annak ellenőrzése, hogy egy adott könyv (ISBN alapján) jelenleg le van-e foglalva.

- ``countUserReservations()``: Egy adott felhasználó által létrehozott foglalások számának lekérdezése.
- ``selectByISBN()``: Foglalás részleteinek lekérdezése egy adott könyvre (ISBN alapján).
- ``selectMyReservations()``: Egy adott felhasználó által létrehozott foglalások listázása, a foglalás részleteivel együtt.

Használat:

- Ezekkel a metódusokkal lehet kapcsolatba lépni a ``reservations`` táblával és a kapcsolódó táblákkal (pl. ``books``, ``authors``, ``categories``).
- A ``selectMyReservations()`` metódus támogatja az összetett lekérdezéseket join-okkal, csoportosítással és szűréssel.

Függőségek:

- A ``Table`` osztályból származik, amely az alap lekérdezésépítési funkciókat biztosítja.
- Használja a kapcsolódó adatbázistáblákat, mint például: ``reservations``, ``books``, ``authors``, ``categories``, és ``publishers``.

Table.php

Ez a fájl a ``Table`` osztályt definiálja, amely alapot biztosít az adatbázis-lekérdezések felépítéséhez. Olyan metódusokat tartalmaz, amelyek lehetővé teszik SQL lekérdezések (pl. SELECT, INSERT, UPDATE, DELETE, JOIN) dinamikus összeállítását és végrehajtását.

Funkciók:

- Lekérdezésépítés:
 - ``select()``: SELECT lekérdezés összeállítása megadott mezőkkel és táblákkal.
 - ``insert()``: INSERT lekérdezés összeállítása megadott mezőkkel és értékekkel.
 - ``update()``: UPDATE lekérdezés összeállítása megadott mezőkkel és értékekkel.
 - ``delete()``: DELETE lekérdezés összeállítása a megadott táblára.
 - Lekérdezésmódosítók:
 - ``where()``: WHERE feltételek hozzáadása operátorokkal, mint pl. ``=``, ``LIKE``, ``IN``, ``IS NULL``.
 - ``groupBy()``: GROUP BY feltételek hozzáadása a lekérdezéshez.

- ``orderBy()``: ORDER BY feltételek hozzáadása növekvő vagy csökkenő sorrendben.
- ``limit()``: LIMIT feltétel megadása az eredmények számának korlátozására.
- ``innerJoin()`` és ``leftJoin()``: JOIN feltételek hozzáadása az adatok több táblából történő összekapcsolásához.
- Végrehajtás:
 - ``execute()``: A felépített lekérdezés végrehajtása, opcionálisan az eredmények lekérésével.
 - ``toString()``: A lekérdezés szöveges formátumba alakítása hibakereséshez vagy al-lekérdezésekhez.
- Egyéb lehetőségek:
 - Automatikus paraméter kötés előkészített utasításokhoz (prepared statements).
 - A lekérdezési állapot automatikus visszaállítása végrehajtás után, hogy az osztály újra felhasználható legyen.

Használat:

- Származtass ebből az osztályból konkrét táblákhoz tartozó lekérdező osztályokat (pl. ``BooksTable``, ``BorrowingsTable``).
- Használd a rendelkezésre álló metódusokat SQL lekérdezések dinamikus felépítésére és futtatására.

Függőségek:

- ``Database\Connection``: Az adatbázis-kapcsolat létrehozására szolgál a lekérdezések végrehajtásához.

UserTable.php

Ez a fájl a ``UserTable`` osztályt definiálja, amely a ``Table`` osztályból származik, és kifejezetten a felhasználókkal kapcsolatos adatok kezelésére és lekérdezésére biztosít metódusokat.

Funkciók:

- ``selectByUsername()``: Felhasználó adatainak lekérdezése felhasználónév alapján.
- ``rowExists()``: Annak ellenőrzése, hogy létezik-e adott feltételeknek megfelelő sor a ``users`` táblában.
- ``updateToVerified()``: Felhasználó email megerősítési állapotának frissítése.

- ``insertToUser()``: Új felhasználó beszúrása a ``users`` táblába.
- ``numberOfUsers()``: Felhasználók számának lekérdezése az adatbázisból.
- ``selectUserData()``: Részletes felhasználói adatok lekérdezése, beleértve a szerepkör-információkat is.
- ``updateUserData()``: Egy adott felhasználó adatainak frissítése.
- ``deleteUserRow()``: Felhasználó törlése a ``users`` táblából.
- ``selectByEmail()``: Felhasználói adatok lekérdezése email-cím alapján.
- ``changePassword()``: Felhasználó jelszavának frissítése.
- ``allByID()``: Összes felhasználói adat lekérdezése felhasználóazonosító alapján.

Használat:

- Ezekkel a metódusokkal lehet kapcsolatba lépni a ``users`` táblával, például hitelesítés, regisztráció, profilfrissítés és fiókkezelés céljából.
- A ``selectUserData()`` és ``updateUserData()`` metódusok támogatják az összetett lekérdezéseket join-okkal és frissítésekkel.

Függőségek:

- A ``Table`` osztályból származik, amely az alap lekérdezésépítési funkciókat biztosítja.
- A ``Helper`` osztályt használja a bemeneti adatok validálására.

BooksController.php

Ez a fájl a ``BooksController`` osztályt definiálja, amely a könyvekkel kapcsolatos HTTP-kéréseket kezeli. Kapcsolatot teremt a kliens és a ``BooksTable`` adatbázis-lekérdezések között, elvégzi a bemeneti adatok validálását, és megfelelő válaszokat ad vissza.

Funkciók:

- ``getFromDBByParams()``: Könyvek lekérdezése az adatbázisból megadott paraméterek alapján.
- ``countAllBooks()``: A könyvek teljes számának visszaadása az adatbázisból.
- ``availableBooks()``: Az éppen elérhető (nem kölcsönzött vagy lefoglalt) könyvek listájának lekérdezése.

Használat:

- Ezt a kontrollert használd könyvekkel kapcsolatos műveletekhez, például kereséshez, számláláshoz vagy elérhetőség ellenőrzéséhez.
- A bemeneti adatokat a `Helper` és a `Books` validáló osztályok segítségével ellenőrzi, mielőtt lekérdezést végezne.

Függőségek:

- `ApiResponse\Response`: HTTP válaszok küldésére szolgál.
- `Database\Queries\BooksTable`: A könyvekhez kapcsolódó adatbázis-lekérdezési metódusokat biztosítja.
- `Helper\Helper`: A bemeneti adatok validálására szolgál.
- `App\Validations\Books`: Könyvekkel kapcsolatos speciális validálási metódusokat tartalmaz.

UserController.php

Ez a fájl a `UserController` osztályt definiálja, amely a felhasználókezeléssel kapcsolatos HTTP-kéréseket kezeli. Kapcsolatot teremt a kliens és a `UserTable` adatbázis-lekérdezések között, elvégzi a bemeneti adatok validálását, hitelesítést, és megfelelő válaszokat ad vissza.

Funkciók:

- `login()`: Felhasználó hitelesítése felhasználónév és jelszó alapján, majd token visszaadása.
- `verifyUser()`: Felhasználói token ellenőrzése.
- `register()`: Új felhasználó regisztrálása, adatok validálása és megerősítő e-mail küldése.
- `verifyAccount()`: Felhasználói fiók megerősítése egy megerősítő kód segítségével.
- `allUsers()`: A felhasználók számának lekérdezése az adatbázisból.
- `userData()`: Felhasználói adatok lekérdezése azonosító alapján.
- `updateUser()`: Felhasználói adatok frissítése azonosító alapján validálás után.
- `deleteUser()`: Felhasználó törlése azonosító alapján.
- `forgotPassword()`: Jelszó-visszaállító e-mail küldése a felhasználónak.
- `changePassword()`: Jelszó megváltoztatása validálás után.

- ``finalizeRegistration()``: Regisztráció véglegesítése e-mail frissítésével és megerősítő e-mail küldésével.

Használat:

- Ezt a kontrollert használd felhasználókkal kapcsolatos műveletekhez, mint például bejelentkezés, regisztráció, profilfrissítés vagy jelszókezelés.
- A bemeneti adatokat a ``Helper`` és ``User`` validáló osztályok ellenőrzik, mielőtt adatbázis-lekérdezés történik.

Függőségek:

- ``ApiResponse\Response``: HTTP válaszok küldésére szolgál.
- ``Database\Queries\UserTable``: A felhasználókkal kapcsolatos adatbázis-lekérdezési metódusokat biztosítja.
- ``Helper\Helper``: Bemeneti adatok validálására szolgál.
- ``App\Validations\User``: Felhasználókkal kapcsolatos speciális validálási metódusokat tartalmaz.
- ``App\Authorize\Token``: Tokenek generálásáért és ellenőrzéséért felelős.
- ``Emailer\SendEmail`` és ``Emailer>EmailBodies``: E-mailek küldésére és tartalomgenerálásra szolgál.

Response.php

Ez a fájl a ``Response`` osztályt definiálja, amely hasznos metódusokat biztosít HTTP válaszok küldésére JSON formátumban. Az osztály célja az API végpontok hibás és sikeres válaszainak egységes kezelése.

Funkciók:

- ``httpError($code, $message)``: HTTP hibaválasz küldése megadott státuszkóddal és hibaüzenettel.
- Ha az üzenet numerikus, akkor a megfelelő hibaüzenetet az ``errorCodes.txt`` fájlból szerzi be.
- ``httpSuccess($code, $json)``: HTTP sikeres válasz küldése megadott státuszkóddal és JSON adatokkal.

Használat:

- Használjuk a ``httpError()`` metódust hibás válaszok küldésére a megfelelő HTTP státuszkóddal és hibaüzenetekkel.
- Használjuk a ``httpSuccess()`` metódust sikeres válaszok küldésére JSON formátumban.

Függőségek:

- Az ``errorCodes.txt`` fájlra támaszkodik, amely a numerikus hiba kódokat a hozzájuk tartozó hibaüzenetekre térképezi.

Megjegyzések:

- Mindkét metódus automatikusan beállítja a ``Content-Type`` fejléct ``application/json`` értékre, és a válasz küldése után lezárja a szkriptet.

User.php

Ez a fájl a ``User`` osztályt definiálja, amely validálási metódusokat biztosít a felhasználókkal kapcsolatos adatok számára. Biztosítja a felhasználói adatok integritását és helyességét, például jelszavak, felhasználónevek, e-mailek és egyéb felhasználói attribútumok esetén. Ezen kívül kezel autentikációs és megerősítési logikát is.

Funkciók:

- ``checkPassword()``: Validálja és hasheli a jelszót, biztosítva, hogy megfeleljen a biztonsági követelményeknek.
- ``checkUsername()``: Validálja a felhasználónevet, és ellenőrzi annak egyediségét az adatbázisban.
- ``checkEmail()``: Validálja az e-mail címet, és ellenőrzi annak egyediségét az adatbázisban.
- ``checkFirstLastName()``: Validálja a kereszt- és családi neveket a megfelelő formázás érdekében.
- ``loginAuth()``: Hitelesíti a felhasználót a felhasználónév és jelszó ellenőrzésével.
- ``createAuthCode()``: Generál egy 6 számjegyű hitelesítő kódot.
- ``checkAuthCode()``: Validálja a hitelesítő kódot az e-mail megerősítése érdekében.
- ``updateToVerified()``: A felhasználót megerősítettre jelöli a sikeres e-mail megerősítés után.

- ``userPasswordIsMatch()``: Ellenőrzi, hogy a megadott jelszó megegyezik-e az adatbázisban tárolt hash-elt jelszóval.
- ``validateAddress()``: Validálja a felhasználói címet a megfelelő formázás és hosszúság érdekében.
- ``validateEmail()``: Validálja az e-mail cím formátumát.
- ``validateDateOfBirth()``: Validálja a felhasználó születési dátumát, hogy megfeleljen az életkori követelményeknek.
- ``isUserVerified()``: Ellenőrzi, hogy a felhasználó megerősítve van-e az e-mail cím vagy a fiók státusza alapján.

Használat:

- Használjuk ezeket a metódusokat a felhasználói adatok validálására regisztráció, bejelentkezés és profilfrissítés során.
- Az olyan metódusok, mint a ``loginAuth()`` és ``checkAuthCode()``, alapvetőek az autentikációs és megerősítési folyamatokban.

Függőségek:

- ``ApiResponse\Response``: HTTP hibás vagy sikeres válaszok küldésére szolgál.
- ``Helper\Helper``: Bemeneti adatok validálására szolgáló segédfunkciók biztosítása.
- ``Database\Queries\UserTable``: Adatbázis-interakciók a felhasználói adatok ellenőrzésére és frissítések végrehajtására.

Model.php

Ez a fájl a ``Model`` osztályt definiálja, amely segédfunkciókat biztosít az adatok validálásához és feldolgozásához. Az osztály alapértelmezettként szolgál az input validálásához, típuskezeléshez és egyéb gyakori műveletekhez, amelyek szükségesek a validálási munkafolyamatokhoz.

Funkciók:

- ``checkRequiredData()``: Validálja és kinyeri a szükséges és opcionális adatokat egy bemeneti tömbből.
- ``makeTypesArray()``: SQL lekérdezésekhez szükséges adattípusok tömbjét generálja a bemeneti mezők és operátorok alapján.
- ``callingValidateFunctions()``: Dinamikusan hívja meg a validálási funkciókat a megadott mezők számára.

- ``validateISBN()``: Validálja az ISBN formátumot (10 vagy 13 számjegy).
- ``validateID()``: Validálja egy azonosítót, hogy csak numerikus karaktereket tartalmazzon.
- ``removeNullValues()``: Eltávolítja a null vagy üres értékeket egy bemeneti tömbből.

Használat:

- Használja ezt az osztályt alapként a validálási logikában más osztályok számára.
- Az olyan metódusok, mint a ``checkRequiredData()`` és ``callingValidateFunctions()``, hasznosak az input adatok strukturált feldolgozásához és validálásához.

Függőségek:

- ``ApiResponse\Response``: HTTP hibás válaszok küldésére szolgál, amikor a validálás nem sikerül.

Image.php

Ez a fájl az ``Image`` osztályt definiálja, amely validálási funkciókat biztosít a képfájlok kezelésére. Biztosítja, hogy a feltöltött képek megfeleljenek bizonyos követelményeknek, mint például fájl típus, méret és létezés.

Funkciók:

- ``validateImage($img)``: Validálja a feltöltött képfájlt a létezés, típus és méret szempontjából.
- ``validateExtentions($fileType)``: Validálja a fájl típusát, és biztosítja, hogy az engedélyezett képformátumok egyike legyen (pl. PNG, JPG, JPEG).
- ``checkForValidFile($imgPath)``: Ellenőrzi, hogy egy érvényes képfájl létezik-e a megadott elérési úton, és a helyes MIME típusú fájlt szolgáltatja.

Használat:

- Használja a ``validateImage()`` metódust a feltöltött képek validálására a fájl feltöltési folyamatok során.
- Használja a ``checkForValidFile()`` metódust képfájlok kiszolgálására a szerverről, ha léteznek.

Függőségek:

- ``ApiResponse\Response``: HTTP hibás válaszok küldésére szolgál, amikor a validálás nem sikerül.

Megjegyzések:

- A maximálisan engedélyezett kép mérete 70 MB.
- Támogatott képformátumok: PNG, JPG, és JPEG.

Books.php

Ez a fájl a ``Books`` osztályt definiálja, amely validálási funkciókat biztosít a könyvekkel kapcsolatos adatok kezelésére. Biztosítja a bemenetek integritását és helyességét, például könyvcímek, kiadási évek, kiadók és szerzők nevének formátumát.

Funkciók:

- ``validateFullName($fullName)``: Validálja egy szerző teljes nevének formátumát.
- ``validateTitle($title)``: Validálja egy könyv címének formátumát.
- ``validateYear($year)``: Validálja egy könyv kiadási évét, biztosítva, hogy az ne legyen a jövőben.
- ``validatePublisher($publisher)``: Validálja egy kiadó nevének formátumát.

Használat:

- Használja ezeket a metódusokat könyvekkel kapcsolatos bemenetek validálására adatrögzítés vagy frissítés során.
- Ezek a validálások biztosítják, hogy az adatok az elvárt formátumoknak és korlátozásoknak megfelelőek.

Függőségek:

- ``ApiResponse\Response``: HTTP hibás válaszok küldésére szolgál, amikor a validálás nem sikerül.

Borrowing.php

Ez a fájl a ``Borrowing`` osztályt definiálja, amely validálási funkciókat biztosít a kölcsönzéssel kapcsolatos adatok kezelésére. Biztosítja a bemenetek helyességét, például a kölcsönzés dátumainak és a kölcsönzési limiteknek.

Funkciók:

- ``checkFormatDate($date)``: Validálja egy dátum formátumát (YYYY-MM-DD), és biztosítja, hogy érvényes naptári dátum legyen.
- ``checkLimit($limit)``: Validálja, hogy a kölcsönzési limit numerikus érték-e.

Használat:

- Használja ezeket a metódusokat kölcsönzéssel kapcsolatos bemenetek validálására adatrögzítés vagy frissítés során.
- Ezek a validálások biztosítják, hogy az adatok az elvárt formátumoknak és korlátozásoknak megfelelőek.

Függőségek:

- ``ApiResponse\Response``: HTTP hibás válaszok küldésére szolgál, amikor a validálás nem sikerül.

ReservationController.php

Ez a fájl a ``ReservationController`` osztályt definiálja, amely a könyv foglalásokkal kapcsolatos HTTP kéréseket kezeli. Közvetítőként működik a kliens és a ``ReservationTable`` adatbázis lekérdezések között, elvégezve a bemenetek validálását és a megfelelő válaszokat visszaadva.

Funkciók:

- ``store($body, $userID)``: Új foglalást hoz létre egy könyv számára egy adott felhasználó által a bemenet validálása után.
- ``destroy($ISBN, $userID)``: Törli egy adott könyv foglalását egy adott felhasználó számára.
- ``show($body, $userID)``: Lekérdezi az összes foglalást, amit egy adott felhasználó tett.

Használat:

- Használja ezt az kontrollert foglalásokkal kapcsolatos műveletekhez, mint például a foglalás létrehozása, törlése és listázása.
- A bemeneti adatokat a `Helper` és `Model` validáló osztályok segítségével validálják, mielőtt adatbázis lekérdezés történik.

Függőségek:

- `ApiResponse\Response`: HTTP válaszok küldésére szolgál.
- `Database\Queries\ReservationTable`: Adatbázis lekérdezési módszerek biztosítása a foglalásokhoz.
- `Database\Queries\BooksTable`: A könyv létezésének validálásához használatos.
- `Database\Queries\BorrowingsTable`: A kölcsönzés állapotának ellenőrzésére szolgál.
- `Helper\Helper`: A bemenetek validálásához használatos.
- `App\Validations\Model`: További validálási módszereket biztosít.

ImageController.php

Ez a fájl az `ImageController` osztályt definiálja, amely a képfeltöltésekkel és lekérésekkel kapcsolatos HTTP kéréseket kezeli. Validálja a képfájlokat, kezeli a feltöltéseket, és kiszolgálja a képeket a szerverről.

Funkciók:

- `uploadImg()`: Kezeli egy kép fájl feltöltését, validálja azt, és áthelyezi a kijelölt könyvtárba.
- `getImg(\$img)`: Lekéri egy kép fájlt az azonosítója alapján, validálja, és kiszolgálja a kliensnek. Ha a kép hiányzik, egy alapértelmezett képet szolgáltat.

Használat:

- Használja az `uploadImg()`-t a képfeltöltések kezelésére a kliensektől.
- Használja a `getImg()`-t képek lekérésére és kiszolgálására a klienseknek.

Függőségek:

- `App\Validations\Image`: A képfájlok validálására szolgál.
- `ApiResponse\Response`: HTTP válaszok küldésére használatos, ha a validálás vagy a fájl műveletek hibát okoznak.

Megjegyzések:

- A feltöltött képek a `booksImages` könyvtárban kerülnek tárolásra.
- Ha egy kép nem található, egy alapértelmezett "hiányzó" képet szolgáltat.

BorrowingsController.php

Ez a fájl a `BorrowingsController` osztályt definiálja, amely a könyvek kölcsönzéseivel kapcsolatos HTTP kéréseket kezeli. A kliens és a `BorrowingsTable` adatbázis lekérdezések között hidat képez, végrehajtja a bemeneti validálást, és megfelelő válaszokat küld vissza.

Funkciók:

- `allBorrowings(\$body)` : Lekéri az adatbázisból a kölcsönzések teljes számát.
- `show(\$body, \$userID)` : Lekéri egy adott felhasználó által kölcsönzött könyvek listáját.
- `topBorrowedBooks(\$limit)` : Lekéri a leggyakrabban kölcsönzött könyveket, a megadott számú limit alapján.

Használat:

- Használja ezt a kontrollert kölcsönzéssel kapcsolatos műveletekhez, mint például a kölcsönzési statisztikák lekérése vagy felhasználó-specifikus kölcsönzések.
- A bemeneti adatokat a `Borrowing` validálási osztály segítségével validálják, mielőtt lekérdezéseket hajtának végre az adatbázisban.

Függőségek:

- `ApiResponse\Response` : A HTTP válaszok küldésére használatos.
- `Database\Queries\BorrowingsTable` : Kölcsönzésekkel kapcsolatos adatbázis lekérdezési módszereket biztosít.
- `App\Validations\Borrowing` : A kölcsönzéssel kapcsolatos bemeneti adatok validálásához használt osztály.

Token.php

Ez a fájl a ``Token`` osztályt definiálja, amely a JSON Web Tokenek (JWT) létrehozására és érvényesítésére szolgáló módszereket biztosít. A felhasználói hitelesítéshez és autorizációhoz használatos az alkalmazásban.

Funkciók:

- ``makeToken($userid, $username, $expireTime = 3600)``: Generál egy JWT-t egy felhasználó számára a megadott lejáratú idővel.
- ``verifyToken($token)``: Érvényesíti a JWT-t és kinyeri a felhasználói azonosítót belőle.

Használat:

- Használja a ``makeToken()`` metódust az autentikált felhasználók számára történő tokenek generálásához.
- Használja a ``verifyToken()`` metódust a tokenek validálásához és a felhasználói információk lekéréséhez védett útvonalakhoz.

Függőségek:

- ``Firebase\JWT\JWT`` és ``Firebase\JWT\Key``: A JWT-k kódolására és dekódolására használt osztályok.
- ``Config\Env``: Betölti a környezeti változókat, beleértve a JWT titkos kulcsot.
- ``ApiResponse\Response``: HTTP hibaválaszok küldése, ha a token érvényesítése nem sikerül.

Megjegyzések:

- A tokeneket az HS256 algoritmus és egy titkos kulcs segítségével írják alá, amely a környezeti változóknak tárolódik.
- Az ``iss`` állítás az alkalmazás alap URL-jére van állítva, és az ``exp`` állítás meghatározza a token lejáratú idejét.

Router.php

Ez a fájl egy egyszerű routerként szolgál, amely kezeli a beérkező HTTP kéréseket, és továbbítja azokat a megfelelő vezérlőkhöz. Meghatározza az alkalmazás API útvonalait, és összeköti azokat a megfelelő vezérlőmetódusokkal.

Funkciók:

- Kezeli az API végpontok útvonalait, beleértve a felhasználókezelést, könyvkezelést, kölcsönzéseket, foglalásokat és képek kezelését.
- Támogatja a többféle HTTP metódust (GET, POST, PUT, DELETE).
- Központosított helyet biztosít az API útvonalak definiálására és kezelésére.
- 404-es hibát ad vissza, ha a kért útvonal nem található.

Útvonalak:

- ``/api/login``: Felhasználói bejelentkezés kezelése.
- ``/api/register``: Felhasználói regisztráció kezelése.
- ``/api/verify-account``: Fiók megerősítésének kezelése.
- ``/api/forgot-password``: Jelszó-visszaállítási kérések kezelése.
- ``/api/user``: Felhasználói adatok lekérése és frissítése.
- ``/api/finalize-registration``: Regisztráció véglegesítése.
- ``/api/change-password``: Jelszó módosításának kezelése.
- ``/api/delete-user``: Felhasználó törlése.
- ``/api/top-borrowings/{limit}``: Leggyakrabban kölcsönzött könyvek lekérése.
- ``/api/borrowings``: Egy adott felhasználó kölcsönzéseinek lekérése.
- ``/api/available-books``: Elérhető könyvek lekérése.
- ``/api/upload-img``: Képfeltöltés kezelése.
- ``/img/{ISBN}``: Könyv borítóképének lekérése.
- ``/api/books``: Könyvek lekérése paraméterek alapján.
- ``/api/all-books``: Az összes könyv számának lekérése.
- ``/api/all-users``: Az összes felhasználó számának lekérése.
- ``/api/all-borrowings``: Az összes kölcsönzés számának lekérése.
- ``/api/reserve``: Könyvfoglalás kezelése.
- ``/api/test``: Teszt útvonal a kölcsönzési elérhetőség ellenőrzésére.

Függőségek:

- ``Router\Router``: Az útvonalak és kérések kezelése.
- ``ApiResponse\Response``: API válaszok és hibaformázás kezelése.
- Vezérlők (Controllers): Az egyes funkciók kezelése (pl. ``UserController``, ``BooksController`` stb.).

Frontend

Az oldal HTTP kéréseit Axios segítségével kezeli, ami egyszerűbb és biztonságosabb szerveroldali kommunikációt tesz lehetővé. Továbbá a tokenek kezeléséhez a JSON Web Token-t a biztonságos adat továbbításához.

App.jsx:

Az **App.jsx** az alkalmazás fő komponense, amely kezeli az útválasztást (*react-router-dom*), az állapotkezelést (*useState*) és az autentikációt (**AuthProvider**).

Az alkalmazás a *react-router-dom* **BrowserRouter**, **Routes** és **Route** komponenseit használja az alábbi útvonalak kezelésére:

Útvonal (Route)	Komponens	Leírás
/	Home	Kezdőlap, statisztikák
/login	Login	Bejelentkezési felület
/forgot-password	ForgotPassword	Elfelejtett jelszó kezelése
/change-password/*	ChangePassword	Jelszó módosítása emailben kapott linkkel
/register	Register	Regisztrációs felület
/books	Books	Könyvek böngészése és szűrése
/finalize-registration	FinalizeRegistration	Előregisztrált felhasználók véglegesítése
/verify	Verify	Email cím megerősítése
/dashboard	Dashboard (PrivateRoute)	Felhasználói irányítópult, csak bejelentkezett felhasználóknak
/dashboard/borrowings	Borrowings	Felhasználó kölcsönzéseinek listája
/dashboard/reservations	Reservations	Foglalások listája, kezelése
/*	NotFound	404-es hibaoldal nem létező útvonalnál

Állapotkezelés (useState):

`searchQuery` állapot: A **Books** komponensben használt keresési kulcsszó, amely a fejlécben (**Header**) megadott keresősáv származik.

- Kezelőfüggvény: `handleSearch(e)` frissíti az állapotot a beviteli mező (*input*) értékével.

```
const [searchQuery, setSearchQuery] = useState("");

function handleSearch(e) {
  setSearchQuery(e.target.value);
}
```

ábra 150: Keresési érték mentése

Autentikáció és Biztonság:

- **AuthProvider**: Egyedi kontextus, amely kezeli a felhasználói hitelesítését (pl.: token ellenőrzés, bejelentkezési állapot).
- **PrivateRoute**: Magasabb rendű komponens, amely védett útvonalakat biztosít, Ha a felhasználó nincs bejelentkezve, átirányít a bejelentkezési oldalra.

```
<Route path="/dashboard" element={<PrivateRoute><Dashboard /></PrivateRoute>}>
  <Route path="borrowings" element={<Borrowings />} />
  <Route path="reservations" element={<Reservations />} />
  <Route path="dashboard" element={<DashboardLogin />} />
  <Route path="profile" element={<Profile />} />
</Route>
```

ábra 151: Tokennel védett útvonalak

AuthProvider.jsx

Az **AuthProvider.jsx** egy kontextus komponens, amely globálisan kezeli a felhasználói autentikációt és munkamenetet.

- Token alapú hitelesítés (JWT – JSON Web Token).
- Felhasználói állapot kezelése (bejelentkezés, kijelentkezés, session lejárt).
- Automatikus token érvényesség ellenőrzése oldalfrissítésekor.

Állapotkezelés (useState):

`user` állapot: Típusa **object**, a bejelentkezett felhasználó dekódolt JWT adatait tárolja (pl.: `userId`, `email`). Ha **null**, a felhasználó nincsen bejelentkezve.

Metódusok:

1. useEffect – Token ellenőrzés

- Az oldal betöltésekor ellenőrizni a **sessionStorage**-ban tárolt JSON Web Token érvényességét.
- Ha token lejárt, törli a tárolt token-t és frissíti az oldalt.
- Hibás token esetén törli a tárolt adatot.

```
useEffect(() => {
  const token = sessionStorage.getItem("token");
  if (token) {
    try {
      const decoded = jwtDecode(token);
      if (decoded.exp * 1000 > Date.now()) {
        setUser(decoded);
      } else {
        alert("Session expired. Please log in again.");
        sessionStorage.removeItem("token");
        window.location.reload();
        setUser(null);
      }
    } catch (error) {
      console.error("Invalid token:", error);
      sessionStorage.removeItem("token");
      setUser(null);
    }
  }
}, []);
```

ábra 152: Token kezelése/dekódolása

2. Login (token) – Bejelentkezés

- Paraméter: JSON web token.
- Elmenti a token-t a **sessionStorage**-ba.
- Dekódolja a token-t (**jwtDecode**) és beállítja a *user* állapotát.

```
const login = (token) => {
  sessionStorage.setItem("token", token);
  const decoded = jwtDecode(token);
  setUser(decoded);
}
```

ábra 153: Bejelentkezés metódusa

3. Logout () – Kijelentkezés

- Törli a token **sessionStorage**-ból.
- Frissíti az oldalt, hogy törölje a cache memóriában maradt állapotokat.
- A **user** állapot **null**-ra állítja.

```
const logout = () => {  
  sessionStorage.removeItem("token");  
  window.location.reload();  
  setUser(null);  
}
```

ábra 154: Kijelentkezés metódusa

setAuthToken.jsx

A **setAuthToken.jsx** egy olyan modul, amely az *Axios HTTP kliens* globális konfigurációját kezeli a JSON Web Token alapú hitelesítéshez.

- Beállítja az **Authorization** fejlécét minden *Axios* kéréshez, ha a felhasználó be van jelentkezve.
- Törli a fejlécet, ha nincs érvényes token.

PrivateRoute.jsx

A **PrivateRoute.jsx** egy olyan komponens, amely védett útvonalakat valósít meg. **Csak bejelentkezett felhasználók számára engedélyezi az elérést.**

- Ha a felhasználónak van érvényes JSON Web Token-je a **sessionStorage**-ban, akkor megjeleníti a gyermek komponens (jelen esetben a */dashboard*-ot).
- Ha nincs token, átirányítja a */login* oldalra.

```
export default function PrivateRoute({ children }) {  
  const isAuthenticated = !!sessionStorage.getItem("token");  
  
  return isAuthenticated ? children : <Navigate to="/login" />;  
}
```

ábra 155: Token által védett útvonal

Header.jsx

A **Header.jsx** egy dinamikus fejléc komponens, amely a következőket biztosítja:

- Navigációs menü a főbb oldalak között (kezdőlap, könyvek, bejelentkezés, regisztráció, irányítópult).
- Keresési sáv a könyvek szűréséhez (csak a / és /books útvonalakon jelenik meg).
- Autentikáció-érzékeny elemek (bejelentkezés/kijelentkezés, irányítópult elérés).
- Reszponzív viselkedés

Főbb funkciók

Keresési sáv: Egy keresési mező, amely alapján lehet szűrni a könyvek között.

- Csak /books útvonalon látható.
- A **hiddenSearch** osztályt a location.pathname alapján állapítja meg.
- A **searchQuery** és **onSearch** prop-ok az **App.jsx**-ből érkeznek, és a könyvek szűréséhez használatosak.

```
<input
  type="text" value={searchQuery || ""}
  onChange={(e) => onSearch(e)}
  placeholder="Search books..."
  className={hiddenSearch}
/>
```

ábra 156: Keresési érték az input mezőben

Navigációs linkek: Kezeli a ki- és bejelentkezett felhasználó láthatóságát

- Kijelentkezett felhasználóknak:

```
{!isAuthorized && <Link to="/register" className="signup">Register</Link>}
{!isAuthorized && <Link to="/login" className="signin">Login</Link>}
```

ábra 157:: Kijelentkezett felhasználók számára látható gombok

- Bejelentkezett felhasználóknak:

```
{isAuthorized && <Link to="/" className="logout" onClick={logout}>Logout</Link>}
{isAuthorized && <Link to="/dashboard" className="dashboard">Dashboard</Link>}
```

ábra 158: Bejelentkezett felhasználók számára látott gombok

- A **/books** útvonal mindkettő esetben látszódik.

Autentikáció kezelése:

- A **logout** függvény az **AuthContext**-ből származik, és törli a token-t a **sessionStorage**-ból.
- Az **isAuthorized** a **sessionStorage**-ban lévő token alapján dönti el a bejelentkezés állapotát.

```
const isAuthorized = !!sessionStorage.getItem("token");
const { logout } = useContext(AuthContext);
```

ábra 159: Token alapú kijelentkezés

TopBorrowings.jsx

A **TopBorrowings.jsx** egy olyan komponens, amely a legtöbbet kölcsönzött könyvek listáját jeleníti meg. Az adatokat egy **REST API** végpontról szerzi be (**/api/top-borrowings/5**), és kártyák formájába rendeli a **TopCards** komponens segítségével. A végpont végén látható szám bármikor megváltoztatható.

```
useEffect(() => {
  const getData = async () => {
    try {
      const response = await axios.get(`/api/top-borrowings/5`);
      console.log(response.data);
      setBooks(response.data);
    } catch (error) {
      console.error("Error fetching books:", error);
      setError("Failed to fetch data");
    } finally {
      setLoading(false);
    }
  };
  getData();
}, []);
```

ábra 160: Az 5 legtöbbet kikölcsönzött könyvet kérdezi le

A komponens a következő elemeket jeleníti meg:

- Cím: „Top Borrowed Books”.
- Hibaüzenet: Ha az API meghívása sikertelen.

- Betöltési állapot: „Loading books...” szöveg, amíg az adatok lekérdezésre kerülnek.
- Könyvkártyák: A **TopCards** komponensek listája a *books* tömb alapján

```
<div children="slider-container">
  <h2 className="slider-title">Top Borrowed Books</h2>
  {error && <div className="error-message">{error}</div>}
  {loading ? (
    <p>Loading books...</p>
  ) : (
    <div id="browsebooks" className="slider-cards-list">
      {books.map((item, index) => (
        <TopCards
          key={index}
          isbn={item.ISBN}
          category={item.Category}
          title={item.Title}
          author={item.Authors}
        />
      ))}
    </div>
  )}
</div>
```

ábra 161: Könyvek listázása

TopCards.jsx

A **TopCards.jsx** egy olyan komponens, amely a legtöbbet kölcsönzött könyvek vizuális megjelenéséért felel, kártyák formájában. Minden kártya megjeleníti a könyv borítóképét, címét, szerzőjét, és kategóriáját.

MainCardContainer.jsx

A **MainCardContainer.jsx** egy fő statisztikai komponens, amely összegző kártyákat jelenít meg a könyvtári rendszer három kulcsfontosságú területéről:

- Könyvek (*/api/all-books*)
- Tagok (*/api/all-users*)
- Kölcsönzések (*/api/all-borrowings*)

A komponens három párhuzamos API hívást kezel, és külön kezeli az egyes adatkészletek betöltési állapotát és hibáit.

A komponens három független *useEffect* hook-ot használ az adatok lekérdezéséhez, például:

```

useEffect(() => {
  const fetchBooks = async () => {
    try {
      const response = await axios.get(`/api/all-books`);
      setBooks(response.data);
    } catch (error) {
      console.error("Error fetching books:", error);
      setError((prev) => ({ ...prev, books: "Failed to fetch books data" }));
    } finally {
      setLoading((prev) => ({ ...prev, books: false }));
    }
  };
  fetchBooks();
}, []);

```

ábra 162: Az adatbázisban található könyvek számát kérdezi le

Cards.jsx

A **Cards.jsx** egy olyan komponens, amely a kölcsönözhető könyvek kártyás megjelenítését biztosítja. Minden kártya tartalmazza a könyv borítóképét, kategóriáját, címét, szerzőjét és egy *Reserve* (Foglalás) gombot. A komponenst a könyvböngészési oldalon használatos (**Books.jsx**).

FormCards.jsx

A **FormsCards.jsx** egy olyan űrlap komponens, amely bevitelmezőke jelenít meg. A komponens kifejezetten modularitásra és újra felhasználhatóságra lett tervezve, így különböző űrlapokban.

DashboardHeader.jsx

A **DashboardHeader.jsx** egy fejléc komponens, amely kizárólag a bejelentkezett felhasználók irányítópultján jelenik meg. A komponens két részből áll:

- Felhasználó üdvözlő fejléc
- Navigációs menü

A komponens a *sessionStorage*-ban mentett token értékét dekódolja, és abból nyeri ki a felhasználó nevét:

```
useEffect(() => {
  const token = sessionStorage.getItem('token');
  if (token) {
    const decodedToken = jwtDecode(token);
    setUser(decodedToken["sub"]);
  } else {
    navigate('/login');
  }
}, [navigate, logout]);
```

ábra 163: Token felhasználása a DashboardHeaderben

4 fő útvonalat tartalmaz:

```
<nav className="navigation">
  <Link className="link" to="/dashboard/borrowings">Borrowings</Link>
  <Link className="link" to="/dashboard/reservations">Reservations</Link>
  <Link className="link" to="/dashboard/dashboard">Dashboard</Link>
  <Link className="link" to="/dashboard/profile">Profile</Link>
</nav>
```

ábra 164: Elérhető útvonalak az irányítópultban (Dashboard)

Books.jsx

A **Books.jsx** a könyvtári rendszer olyan komponensre, amely lehetővé teszi a felhasználók számára:

- Az elérhető könyvek böngészését,
- Szűrést szerző, kiadó, év és kategória alapján,
- Keresést cím és szerző alapján,
- Könyvek foglalását,
- Szűrők visszaállítását.

A komponens hatékonyan kezeli a nagy mennyiségű könyv adatok megjelenítését és szűrését a `useMemo` hook segítségével.

Állapotkezelés:

books: Az összes elérhető könyv listája (`/api/available-books`).

loading: Betöltési állapot.

filters: author, publisher, year, category.

searchQuery: Keresési kulcsszó.

Borrowings.jsx

A **Borrowings.jsx** komponens a felhasználó által kölcsönzött könyvek listáját jeleníti meg táblázatos formában. Csak a bejelentkezett felhasználók számára elérhető, és az irányítópult részeként működik.

- Kölcsönzött könyvek listájának lekérése API-ból (/api/avaiable-books),
- Táblázatos megjelenítés ISBN, cím szerző kölcsönzés dátuma és határidő alapján,
- Autentikációs token használata.

ChangePassword.jsx

A **ChangePassword.jsx** komponens lehetővé teszi a felhasználók számára jelszavuk megváltoztatását egy biztonságos folyamaton keresztül.

- Token kinyerése az URL-ből
- Új jelszó és megerősítés bekérése
- Jelszó változtatás API (/api/change-password) hívása
- Hibakezelés
- Sikeres változás esetén átirányítás a bejelentkezési oldalra

Dashboard.jsx

A **Dashboard.jsx** a könyvtári rendszer irányítópultja, amely a felhasználói felület összes alpaneljét/komponensét tartalmazza. Ez a komponens szolgál konténerként a következő alkomponensek számára:

- DashboardHeader: Navigációs fejléc
- Borrowings: Kölcsönzött könyvek listája
- Reservations: Foglalások kezelése
- Profile: Felhasználói profil adatok

FinalizeRegistration.jsx

A **FinalizeRegistration.jsx** komponens kezeli a felhasználói regisztráció véglegesítésének folyamatát, ahol az előzetesen regisztrált felhasználók megadhatják email címüket. A felhasználónevet és jelszót azt az asztali alkalmazásban, azaz a könyvtárostól kapja meg ha fizikálisan regisztrál a felhasználó. A komponens egy űrlapot jelenít meg, ami elküldi az adatokat a szervernek, majd átirányítja a verifikációs oldalra.

- Email, felhasználónév és jelszó megadása
- Jelszó megjelenítése/elrejtése
- Adatok elküldése a backend PUT (/api/finalize-registration) kéréssel
- Hibakezelés
- Sikeres regisztráció esetén átirányítás a verifikációs oldalra

ForgotPassword.jsx

A **ForgotPassword.jsx** komponens lehetővé teszi a felhasználók számára, hogy elfelejtett jelszavukat visszaállítsák. A komponens egy űrlapot jelenít meg, ahol felhasználó megadhatja email címét, amire a rendszer elküldi a jelszó-visszaállítási linket.

- Email cím bekérése
- Jelszó visszaállító email küldése
- Sikeres küldés visszajelzése
- Hibakezelés
- Elfelejtett jelszó API (/api/forgot-password) hívása

Home.jsx

A **Home.jsx** komponens a rendszer kezdőlapját jeleníti meg, amely kettő szekciót jelenít meg, *Hero* és *Main* komponensek. Ez a komponens fogadja a felhasználókat elsődleges belépési pontjaként.

Login.jsx

A **Login.jsx** komponens a bejelentkezési felületet valósítja meg. A komponens lehetővé teszi a felhasználók számára, hogy felhasználónév és jelszó megadásával belépjenek a rendszerbe, és kezel minden kapcsolódó állapotot, hibát és hitelesítési folyamatot.

- Felhasználónév és jelszó alapú bejelentkezés
- Jelszó megjelenítése/elrejtése funkció
- Hibakezelés
- Token mentése *sessionStorage*-ba
- Sikeres bejelentkezés esetén átirányítás a dashboard-ra

Űrlap elküldése:

- Megakadályozza az üres űrlap elküldését
- Beállítja a *loading* állapotot *true*-ra
- Törli a korábbi hibákat

API hívás:

- Küld egy *POST* kérést a */api/login* végpontra
- Az adat tartalmazza a felhasználónevet és jelszót
- A válasz alapján:
 - Sikeres bejelentkezés esetén:
 - Beállítja az auth token
 - Elmenti a token a *sessionStorage*-ba
 - Átirányít a */dashboard* oldalra

NotFound.jsx

A **NotFound.jsx** komponens egy 404-es hibaoldal, amely akkor jelenik meg, amikor a felhasználó olyan útvonalra navigál, amely nem létezik a könyvtári rendszerben.

- Statikus komponens
- Nincs külső függőség
- Minden nem definiált útvonalat hibaként érzékel
- Egyszerű visszajelzést ad a felhasználónak

Profile.jsx

A **Profile.jsx** komponens a felhasználói profil kezeléséért felelős, ahol a felhasználók:

- Megtekinthetik a személyes adataikat
- Módosíthatják profiljukat
- Megváltoztathatják jelszavukat
- Törölhetik a felhasználójukat

A komponens csak a `/dashboard/profile` útvonalon érhető el, és JSON Web Token alapú hitelesítést igényel.

Register.jsx

A **Register.jsx** komponens egy regisztrációs űrlapot jelenít meg. Feladata a felhasználói adatok (név, email, jelszó, születési dátum, cím) begyűjtése és elküldése a backend-nek.

Űrlap elküldése:

- Megakadályozza az üres űrlap elküldését
- Beállítja a *loading* állapotot *true*-ra
- Törli a korábbi hibákat

API hívás:

- Küld egy *POST* kérést a `/api/register` végpontra
- Az adatokat JSON formátumban kerülnek elküldésre
- A válasz alapján:
 - Sikeres regisztráció esetén átirányít a `/verify` oldalra
 - Sikertelen regisztráció esetén megjeleníti a hibát

Az adatok validációját a backend végzi.

Reservations.jsx

A **Reservations.jsx** egy olyan komponens, amely egy felhasználó által fogalt könyvek listáját jeleníti meg táblázatos formában. A komponens lekéri a szerverről a felhasználó foglalásait, és megjeleníti azokat ISBN, cím, szerző, foglalás kezdete, foglalás vége és művelet alapján.

API hívás:

- Küld egy *GET* kérés a `/api/reserve` végpontra
- Az *Authorization* header tartalmazza a felhasználó tokenjét

Verify.jsx

A **Verify.jsx** egy olyan komponens, amely kezeli a felhasználói fiók ellenőrzését és mind az újonnan regisztrált felhasználók, mind az előregisztrált felhasználókat véglegesíti. A komponens egy ellenőrző kód bekérése szolgáló űrlapot jelenít meg, majd elküldi a kódot a szervernek validálásra.

Űrlap elküldése:

- Megakadályozza az üres űrlap elküldését
- Beállítja a *loading* állapotot *true*-ra
- Törli a korábbi hibákat

API hívás:

- Küld egy *POST* kérést a */api/verify* végpontra
- Az adat tartalmazza az ellenőrző kódot
- A válasz alapján:
 - Sikeres ellenőrzés átirányít a */login* oldalra
 - Sikertelen ellenőrzés esetén megjeleníti a hibát

Reszponzivitás

A könyvtári rendszer weboldali *reszponzív stílusrendszere*, amelyek a következő komponensek dizájnját tartja számon:

- Globális stílusok (alapértelmezett beállítások)
- Komponens-specifikus stílusok (Login, Register, Dashboard stb.)
- Reszponzív tervezés (telefon, tablet, asztali)
- Dizájn rendszer (színek, tipográfia, árnyékok)

A Globális stílusokért az *index.css* felel.

Dizájn rendszer:

Szín	HEX	Használat
Primary	#504DB4	Fő szín, gombok, fejléc
Secondary	#48459E	Másodlagos szín, hover állapotok
White	#FFFFFF	Háttér, szöveg
Black	#000000	Szöveg, keretek
Error	#FF0000	Hibaüzenet

Tipográfia

- Betűtípus: Rendszer alapértelmezett (sans-serif)
- Méretek:
 - Fejléc: *2rem – 5rem*
 - Szövegek: *1rem – 1.5rem*
 - Gombok: *1.2rem*

Komponens-specifikus stílusok:

A komponens úgy lett tervezve, hogy különböző oldalakon eltérő elrendezéseket átláthatóbban lehet szerkeszteni, bővíteni, továbbá dinamikusan kezelje az egyes elemek láthatóságát. Alkalmazkodjon a változó méretű beviteli mezők igényeihez minden egyes oldalon. Ez lehetővé teszi, hogy minden felületrész testreszabott megjelenést és viselkedést kapjon, miközben a komponens maga újra felhasználható maradjon.

- Login.css
- Register.css
- Verify.css
- SliderColousel.css
- Dashboard.css

Reszponzív tervezés:

Eszköz	Szélesség	Példa
Mobil	< 768px	@media (max-width: 768px)
Tablet	768px – 1024px	@media (min-width: 768px) and (max-width: 1024px)
Asztali	> 1024px	Alapértelmezett

Frontend Tesztelés:

A tesztek Selenium WebDriverrel készültek, amelyek a webalkalmazás regisztrációs és bejelentkezési folyamatát vizsgálják.

login.test.js:

Először egy ChromeDriver példányt hoz létre, amelyet konfigurál a *–no-sandbox* beállítással, majd megnyitja a bejelentkezési oldalt *localhost-on*. Ezután kitölti a felhasználónév és jelszó mezőket, majd a *Key.RETURN* segítségével elküldi az adatokat. A sikeres bejelentkezés ellenőrzése a rendszer megvárja, hogy a *dashboard* elem megjelenjen az oldalon, amely igazolja a sikeres hitelesítést. Végül a kód terminálba kiírja a sikeres bejelentkezési üzenetet, vagy hiba esetén értesíti a felhasználót, majd bezárja a böngészőt. Ez egy automatizált teszt.

register.test.js:

Először létrehoz egy WebDriver példányt, amely a Chrome böngészőt használja, majd megnyitja a regisztrációs oldalt a *localhost:5173/register* címen. Ezután kitölti a felhasználói adatmezőket, beleértve a vezeté- és keresztnévet, az e-mail címet, a felhasználónevet és a jelszót. A születési dátumot JavaScript segítségével állítja be, amely közvetlenül módosítja az input értékét és eseményt küld annak frissítéséhez. Ezután a rendszer megadja a lakcímet, majd a regisztrációs űrlapot elküldi a megfelelő gomb megnyomásával. Ha a regisztráció sikeres, a script megvárja, hogy az oldal átirányítson a */verify* URL-re, és kiírja a sikeres tesztelés üzenetét. Amennyiben hiba lép fel a folyamat során, azt naplózza, majd végül bezárja a böngészőt, biztosítva a tesztelési folyamat lezárását.

LMS Desktop (asztali alkalmazás)

Az asztali alkalmazás C# Windows Forms-ban készült .NET 8.0-val. Nugget csomagok közül a MySQL-t, illetve a BCrypt-et használ a program. A telepítőkészletet az Inno Setup nevű programmal készítettük el.

Connection osztály

- Az adatbázis adatai bele vannak írva a kódba, nincsenek titkosítva.
- Saját Select függvény, ami egy lekérdezést fogad és visszaad egy string tömbökből álló listát.
- RunSqlCommand függvény, ami a kapott lekérdezést egyszerűen lefuttatja.

HandleFiles osztály

- Egy képfájl feltöltéséért felel a benne lévő Upload függvény.
 - Itt a kép átküldésre kerül a backendnek HttpClient segítségével.
 - Az esetleges hibákat json formátumban küldi vissza a backend.
 - Kivételkezelést is tartalmaz, ahol egyéb hibákat szűrünk ki.

HandleQueries osztály

- Ez az osztály felel az egyes lekérdezések, beszúrások és frissítések megvalósítására.
 - A sima lekérdezések csak egy lekérdezést vagy egy fájlnevet várnak el. A fájlok az SqlQueries mappában található.
 - A Delete függvény egy lekérdezés alapján kitörli kijelölt elemeket az adatbázisból.
 - Az összes többi függvény minden egyes Insert és Update műveletet valósít meg, ahol paraméterekben várja a feltöltendő / frissítendő adatokat.

Egyéb osztályok

- GeneratePassword -> Egy függvényt tartalmaz, ami a megadott karakterek és hossz alapján generál egy véletlenszerű jelszót.
- HandleGrids -> Függvényeket tartalmaz ComboBox és dataGridView feltöltésére, plusz egy megadott griden belüli keresésre.
- BorderPaint -> Keretet rajzol az adott form köré.
- CloseWindow -> Gombhoz rendelhető ablakbezárás, a megadott formot bezárja.
- DragWindow -> Lehetővé teszi az ablakok mozgatását címke (title) és panelek megadásával.

- HandleKeys -> A megadott gombra meghív egy megadott függvényt (pl.: Az Enter lenyomásakor elmenti a beállításokat, Escape gomb megnyomásakor pedig bezárja az ablakot).
- HandleFonts -> Beállítja a megadott form összes elemére a megadott betűtípust.

Login form

- Ez az első oldal, ami az alkalmazás indításakor megjelenik.
- Az alkalmazás az adatbázis nélkül nem fog elindulni.
- A LoginCheck függvényen belül ellenőrzésre kerülnek a felhasználó adatai. Ha a felhasználónév alapján található ilyen felhasználó, akkor BCrypt Verify segítségével összehasonlítjuk az adatbázisban tárolt titkosított jelszót a felhasználó által megadott jelszó titkosított változatával. Ha a két titkosított jelszó egyezik, akkor a jelszavak egyeznek. Így nincs szükség a titkosítás feloldására, ezáltal biztonságosabb a program.
- Ha a felhasználó rossz adatot ad meg, arról tájékoztatja a program.
- A UsernameTextChanged és PasswordTextChanged függvények felelnek a valós idejű hibák kiírására a felhasználónév és jelszóval kapcsolatban.
- A ShowPassword függvény pedig a szem ikont megváltoztatja, és megjeleníti a jelszót.

Main form

- A Main form felel az összes művelet megvalósítására.
- Minden adat kezelhető külön oldalakon (Dashboard, Books, Borrowings, Reservations, Categories, Authors, Publishers és Users).
- Itt kerülnek meghívásra az egyes SQL lekérdezések.
- A címre kattintva a Dashboard nyílik meg
 - Itt külön-külön lekérdezés van a statisztikák kiírtatására (Könyvek, Felhasználók és Kölcsönzések száma)
 - Külön lekérdezés a 10 legtöbbet kölcsönzött könyv kiíratására.
- A jobb felső sarokban található üdvözlő szövegre kattintáskor tudja a felhasználó szerkeszteni saját fiókját.
- A Logout gomb pedig kijelentkezteti a felhasználót, majd megnyitja a Login formot.

Profile Settings

- A form megnyitásakor betöltésre kerülnek a bejelentkezett felhasználó adatai, amiket tud módosítani is.
- A validáció minden adatot ellenőriz, beleértve, ha a felhasználónevét módosítja, akkor ne tudja olyanra módosítani, ami már létezik.
- A validációt követően lefut a UpdateProfile függvény, ami frissíti a felhasználó adatait az adatbázisban.
- A ChangePassword felel a felhasználó jelszavának megváltoztatására.
 - Itt a validáció során ellenőrzésre kerül a Login-ban említett módon, hogy megadott jelenlegi jelszó tényleg megegyezik-e a felhasználó jelszavával. Ezt követően a másik két szövegdoboz is ellenőrzésre kerül.
 - A ShowPassword lehetővé teszi, hogy a felhasználó meg tudja jeleníteni a beírt jelszót.
 - A jelszó írása közben a gombok lenyomására kiírja a felhasználónak, hogy minek és minek nem felel meg a beírt jelszó.
 - Végül a mentésre kattintva a jelszó frissül az adatbázisban (UpdatePassword függvény).

Books

- Az AddBook-ban meg lehet nyitni a ChooseAuthor és ChooseCategory ablakokat, ahol ki lehet választani egy vagy több szerzőt és kategóriát.
- Az adatok validálása után a könyv adatai feltöltésre kerülnek az adatbázisba (InsertBook függvény).
- Opcionálisan képet is lehet feltölteni:
 - A kép kiválasztásakor validálásra kerül a kép, itt ellenőrizve lesz a kép formátuma (png, jpg, jpeg), illetve a kép mérete (Max 5 MB).
 - Az ellenőrzés után át lesz másolva a temp mappába. A fájl neve itt a könyv isbn száma lesz.
 - Ezt követően meghívásra kerül a HandleFiles osztály Upload függvénye.
- Az EditBook szinte teljesen megegyezik az AddBook formával, azzal a különbséggel, hogy ide betöltődnek a kijelölt könyv adatai (UpdateBook függvény).
- A RemoveBook egész egyszerűen eltávolítja a kiválasztott könyve(ke)t az adatbázisból (Delete függvény).

Borrowings

- Az AddBorrowing-ban meg lehet nyitni a könyvek kiválasztására szolgáló ChooseBooks formot, ebből a SelectedISBNs nevű string lista mezőből betöltésre kerülnek az ISBN számok a megfelelő szövegdobozba.
- A validációt követően a dátumok megfelelő formátumba lesznek váltva (InsertBorrowing függvény).
- Az ExtendBorrowing-ban a jelenlegi határidőhöz hozzáadásra kerül a felhasználó által megadott hónapok száma (UpdateBorrowing függvény).
- A ReturnBorrowings-ban a könyv visszahozásának dátuma mindig aznap lesz (UpdateBorrowing overloaded függvény).

Reservations

- Az AddReservation felel egy vagy több foglalás felvételére.
 - Az OpenChooseBooks megnyitja a könyvek kiválasztására szolgáló formot, ami egyébként megegyezik a Borrowings részben használt könyv kiválasztó formmal. Itt, ha több könyvet választ ki a felhasználó, azok külön rekordokként jelennek majd meg az adatbázisban.
 - A mentésre kattintva lefut a validáció, és ha minden rendben van feltöltésre kerül az adatbázisba (InsertReservation függvény).
- Az ExtendReservation form felel egy foglalás meghosszabbítására előre meghatározott 5 nappal. Az adott foglalás lejáratí dátumához hozzáad 5 napot és frissíti azt az adatbázisban (UpdateReservation függvény).
- A LendReservation a kiválasztott foglalás alapján kölcsönbe adja az adott könyvet. A foglalás tartalmazza a felhasználónevet és az ISBN számot, a könyvtáros meg kell adjon egy határidőt, és ezek alapján bekerül a kölcsönzés az adatbázisba (InsertBorrowing függvény), majd törlésre kerülnek a foglalások táblából (Delete függvény).
- A RemoveReservations pedig eltávolítja a kiválasztott foglalás(oka)t az adatbázisból (Delete függvény).

Categories

- Az AddCategory validációja után, beleértve annak ellenőrzését, hogy létezik e már a megadott kategória, az feltöltésre kerül az adatbázisba (InsertCategory).
- Az EditCategory betölti a kiválasztott kategória nevét, majd az előző pontban említett validáció után frissítve lesz az adatbázisban (UpdateCategory).
- A RemoveCategory pedig eltávolítja a kiválasztott kategóriá(ka)t az adatbázisból (Delete függvény).

Authors

- Az AddAuthor-ban a validációt követően feltöltésre kerül a szerző (InsertAuthor függvény).
- Az EditAuthor-ban betöltődik a kiválasztott szerző, majd a validációt követően módosításra kerül (UpdateAuthor függvény).
- A RemoveAuthor a paraméterben kapott szerző(ke)t eltávolítja (DeleteFüggvény).

Publishers

- Az AddPublisher a mentésre kattintást követően végrehajtja a validációt, ami többek között leellenőrzi, hogy a megadott kiadó létezik e már az adatbázisban. Amennyiben a validáció sikeres, a kiadó feltöltésre kerül az adatbázisba (InsertPublisher függvény).
- Az EditPublisher ablakban a kiválasztott kiadó neve betöltésre kerül a szövegdobozba, majd az előző pontban említett validációt követően frissítésre kerül az adatbázisban (UpdatePublisher függvény).
- A RemovePublishers a kiválasztott kiadó(ka)t eltávolítja az adatbázisból (Delete függvény).

Users

- Az AddUser a validációt követően, beleértve azt, hogy a felhasználónév biztosan egyedi e, feltölti a felhasználó adatait az adatbázisba (InsertUser függvény).
 - Kötelező adatok: Vezetéknév, Keresztnév, Felhasználónév, Születési dátum, Lakcím.
 - Kötelező adatok adminként: szerep, ami lehet
 - Member – Sima felhasználó, tud kölcsönözni és foglalni.
 - Librarian – Könyvtáros, tudja kezelni a könyvtári rendszer jelentős részét (kivéve a felhasználók felhasználónevét és rangját).
 - Admin – Adminisztrátor, a teljes rendszerhez hozzáfér.
 - A felhasználó felvételét követően egy véletlenszerűen generált jelszó társul a fiókhoz, amit a webes felületen később meg tud változtatni.
- Az EditUser a kiválasztott felhasználó datainak betöltése után az előző pontban említett validációt követően frissíti a felhasználó adatait az adatbázisban (UpdateUser függvény).
 - Itt ugyan azok az adatok módosíthatóak, viszont csak az Admin tudja megváltoztatni a felhasználónevet vagy a rangot.
 - A felhasználót meg lehet jelölni megerősítettként, ami azt jelenti, hogyha a személyes adatai valósak, csak akkor tud kölcsönözni.
 - A Reset Password (jelszó visszaállítása) gomb arra szolgál, ha a felhasználó elfelejtette vagy elhagyta az ideiglenes jelszavát, akkor a könyvtárosnak lehetősége van újat generálnia számára.
- A RemoveUser pedig eltávolítja a kiválasztott felhasználó(ka)t az adatbázisból (Delete függvény).

SQL Lekérdezések

SelectTopBorrowedBook

- Lekérdezi a leggyakrabban kölcsönzött könyveket, majd csökkenő sorrendbe rendezi a kölcsönzések száma alapján.

SelectBook

- Lekérdezi az összes könyvet, a szerzőket és kategóriákat vesszővel elválasztva csoportosítja, majd rendszerezi a könyv címe alapján ábécé sorrendbe.

SelectAllBorrowing

- Lekérdezi az összes kölcsönzést a Borrowings_storage táblából, ami tartalmazza a törlésre került kölcsönzéseket is. A kölcsönzés dátuma alapján csökkenő sorrendben van rendezve.

SelectCurrentBorrowing

- Lekérdezi a jelenleg aktív kölcsönzéseket, azaz ahol a visszahozási dátum NULL.

SelectReservation

- Lekérdezi a foglalásokat, rendszerezi a foglalás kezdetének dátuma alapján csökkenő sorrendben, azon belül az ISBN szám alapján növekvő sorrendben.

SelectCategory

- Lekérdezi az összes kategóriát.

SelectAuthor

- Lekérdezi az összes szerzőt.

SelectPublisher

- Lekérdezi az összes kiadót.

SelectUser

- Lekérdezi az összes felhasználó adatait, ha meg van erősítve a fiókja akkor „Yes”, ha nincs akkor „No” van kiírva. A rendezés vezeté- és keresztnév alapján történik.

SelectAuthorWithBook

- Lekérdezi az összes szerzőt, majd vesszővel elválasztva kisorolja mellé a szerző könyveit, hogy könnyebben beazonosítható legyen a szerző.

SelectBookForBorrowing

- Lekérdezi az összes könyvet, viszont itt csak a cím, szerzők, kiadás éve és az ISBN szám szerepel.

SelectRole

- Lekérdezi az összes szerepet.

SelectUsername

- Lekérdezi az összes felhasználónevet. Ez akkor hasznos, amikor a felhasználóneveket ellenőrizzük.

SelectBookCount

- Lekérdezi az adatbázisban tárolt könyvek számát.

SelectUserCount

- Lekérdezi az adatbázisban tárolt felhasználók számát.

SelectBorrowingCount

- Lekérdezi az adatbázisban tárolt kölcsönzések számát a Borrowings_storage táblából.

Tesztelés

A tesztelés NUnit-tal van megoldva, egy adott bemeneti mező lehetséges bemeneteit megkapja a teszt, és visszaadja annak eredményét. Ezek a tesztek ellenőrzik, hogy a szoftver képes e megakadályozni az SQL injection-t, illetve nem fogad e el olyan adatokat, amik nem valóságszerűek.

```
[TestCase("", "halo")]
[TestCase("\\"halo\\", "halo")]
[TestCase("halo", "")]
[TestCase("halo", "\\"halo\\"")]
[TestCase("halo", "halo")]
0 references
public void TestInput(string username, string password)
{
    if (string.IsNullOrEmpty(username))
    {
        Assert.Fail("Username is empty");
    }
    else if (Regex.IsMatch(username, @"[^\\"\\]+$"))
    {
        Assert.Fail("Username contains not allowed characters");
    }

    if (string.IsNullOrEmpty(password))
    {
        Assert.Fail("Password is empty");
    }
    else if (Regex.IsMatch(password, @"[^\\"\\]+$"))
    {
        Assert.Fail("Password contains not allowed characters");
    }
    Assert.Pass();
}
```

ábra 165: Bejelentkezés teszt

```
[TestCase("")]
[TestCase("\\"No title\\"")]
[TestCase("Good title")]
0 references
public void TestTitle(string title)
{
    if (title == string.Empty)
    {
        Assert.Fail("Title is required");
    }
    else if (!Regex.IsMatch(title, @"[^\\"\\]+$"))
    {
        Assert.Fail("Title contains not allowed characters");
    }
    Assert.Pass();
}
```

ábra 166: Könyv cím teszt

Források

Backend

- [PHPMailer](#)

Frontend

- [Website icon](#)

LMS Desktop

- [Close icon](#)
- [Refresh icon on Admin Panel](#)
- [Default logo – book icon](#)
- [Eye icon](#)
- [Crossed eye icon](#)
- [Right arrow icon](#)
- [Information icon](#)

Ábrajegyzék

ábra 1: végpont és metódus	ábra 2: kérés törzse	6
ábra 3: válaszüzenet	ábra 4: válasz kódja	6
ábra 5: kérés törzse	ábra 6: válasz kódja	7
ábra 7: válaszüzenet		7
ábra 8: végpont és metódus	ábra 9: válaszüzenet	7
ábra 10: kérés törzse	ábra 11: válasz kódja, ideje és mérete	7
ábra 12: verifikációs e-mail		8
ábra 13: kérés törzse	ábra 14: válaszüzenet	8
ábra 15: válasz kódja, ideje és mérete		8
ábra 16: végpont és metódus		9
ábra 17: válaszüzenet	ábra 18: válasz kódja, ideje és mérete	9
ábra 19: kérés törzse	ábra 20: válaszüzenet	9
ábra 21: válasz kódja, ideje és mérete		9
ábra 22: végpont és metódus		9
ábra 23: kérés törzse	ábra 24: válaszüzenet	10
ábra 25: válasz kódja, ideje és mérete		10
ábra 26: kérés törzse	ábra 27: válaszüzenet	10
ábra 28: válasz kódja, ideje és mérete		10
ábra 29: automatikus e-mail		10
ábra 30: végpont és metódus		11
ábra 31: válaszüzenet	ábra 32: válasz kódja, ideje és mérete	11
ábra 33: végpont és metódus		11
ábra 34: kérés törzse	ábra 35: válaszüzenet	12
ábra 36: válasz kódja, ideje és mérete		12
ábra 37: kérés törzse	ábra 38: válaszüzenet	12
ábra 39: válasz kódja, ideje és mérete		12
ábra 40: végpont és metódus	ábra 41: válaszüzenet	13
ábra 42: válasz kódja, ideje és mérete		13
ábra 43: válaszüzenet	ábra 44: válasz kódja, ideje és mérete	13
ábra 45: végpont és metódus		13
ábra 46: kérés törzse	ábra 47: válaszüzenet	14
ábra 48: válasz kódja, ideje és mérete		14
ábra 49: kérés törzse	ábra 50: válaszüzenet	14
ábra 51: válasz kódja, ideje és mérete		14
ábra 52: verifikációs e-mail		14
ábra 53: végpont és metódus		15
ábra 54: kérés törzse	ábra 55: válaszüzenet	15
ábra 56: válasz kódja, ideje és mérete		15
ábra 57: kérés törzse	ábra 58: válaszüzenet	15
ábra 59: válasz kódja, ideje és mérete		15
ábra 60: végpont és metódus		16
ábra 61: válaszüzenet	ábra 62: válasz kódja, ideje és mérete	16
ábra 63: végpont és metódus		17
ábra 64: válaszüzenet	ábra 65: válasz kódja, ideje és mérete	17

ábra 66: végpont és metódus	18
ábra 67: válaszüzenet ábra 68: válasz kódja, ideje és mérete	18
ábra 69: végpont és metódus ábra 70: válaszüzenet	18
ábra 71: válasz kódja, ideje és mérete	18
ábra 72: kérés törzse	19
ábra 73: válaszüzenet ábra 74: válasz kódja, ideje és mérete	19
ábra 75: végpont és metódus	19
ábra 76: válaszüzenet ábra 77: válasz kódja, ideje és mérete	19
ábra 78: végpont és metódus	20
ábra 79: válaszüzenet (kép formátumban) ábra 80: válasz kódja, ideje és mérete	20
ábra 81: végpont és metódus	20
ábra 82: válaszüzenet (kép formátumban) ábra 83: válasz kódja, ideje és mérete	20
ábra 84: végpont és metódus	21
ábra 85: válaszüzenet ábra 86: válasz kódja, ideje és mérete	21
ábra 87: végpont és metódus	21
ábra 88: válaszüzenet ábra 89: válasz kódja, ideje és mérete	21
ábra 90: végpont és metódus	22
ábra 91: válaszüzenet ábra 92: válasz kódja, ideje és mérete	22
ábra 93: végpont és metódus	22
ábra 94: válaszüzenet ábra 95: válasz kódja, ideje és mérete	22
ábra 96: végpont és metódus ábra 97: kérés törzse	23
ábra 98: válaszüzenet ábra 99: válasz kódja, ideje és mérete	23
ábra 100: végpont és metódus	23
ábra 101: válaszüzenet ábra 102: válasz kódja, ideje és mérete	23
ábra 103: Regisztrációs	27
ábra 104: Töltés közbeni animáció.....	28
ábra 105: Verifikációs kód	28
ábra 106: Fiók visszaigazolása	28
ábra 107: Bejelentkező	29
ábra 108: Kölcsönzések.....	29
ábra 109: Foglalások	30
ábra 110: Fiók frissítése	30
ábra 111: Könyvek	31
ábra 112: Foglalási limit.....	31
ábra 113: Bejelentkező felület ábra 114: Jelszókezelés.....	32
ábra 115: Műszerfal.....	33
ábra 116: Profilbeállítások ábra 117: Jelszóváltoztatás.....	33
ábra 118: Könyvek	34
ábra 119: Könyv hozzáadása ábra 120: Könyv szerkesztése	35
ábra 121: Könyv eltávolítása.....	35
ábra 122: Kölcsönzések.....	36
ábra 123: Kölcsönbe adás ábra 124: Kölcsönzés meghosszabbítása.....	36
ábra 125: Könyvek kiválasztása	37
ábra 126: Könyv visszahozása	37
ábra 127: Foglalások	38
ábra 128: Kölcsönbe adás ábra 129: Foglалás hozzáadása	38
ábra 130: Foglалás meghosszabbítása ábra 131: Foglалás visszamondása	39

ábra 132: Kategóriák	40
ábra 133: Kategória hozzáadása ábra 134: Kategória szerkesztése	40
ábra 135: Kategória eltávolítása	40
ábra 136: Szerzők	41
ábra 137: Szerző hozzáadása ábra 138: Szerző szerkesztése	42
ábra 139: Szerző eltávolítása	42
ábra 140: Kiadók	43
ábra 141: Kiadó hozzáadása ábra 142: Kiadó szerkesztése	43
ábra 143: Kiadó eltávolítása	43
ábra 144: Felhasználók	44
ábra 145: Felhasználó hozzáadása	45
ábra 146: Felhasználó szerkesztése	45
ábra 147: Felhasználó eltávolítása ábra 148: Generált jelszó	45
ábra 149: Adatbázis diagram	46
ábra 150: Keresési érték mentése	70
ábra 151: Tokennel védett útvonalak	70
ábra 152: Token kezelése/dekódolása	71
ábra 153: Bejelentkezés metódusa	71
ábra 154: Kijelentkezés metódusa	72
ábra 155: Token által védett útvonal	72
ábra 156: Keresési érték az input mezőben	73
ábra 157:: Kijelentkezett felhasználók számára látható gombok	73
ábra 158: Bejelentkezett felhasználók számára látott gombok	73
ábra 159: Token alapú kijelentkezés	74
ábra 160: Az 5 legtöbbet kikölcsönzött könyvet kérdezi le	74
ábra 161: Könyvek listázása	75
ábra 162: Az adatbázisban található könyvek számát kérdezi le	76
ábra 163: Token felhasználása a DashboardHeaderben	77
ábra 164: Elérhető útvonalak az irányítópultban (Dashboard)	77
ábra 166: Bejelentkezés teszt ábra 167: Könyv cím teszt	92